TUGAS 6 KELOMPOK 4

(Basisdata NoSQL)

KELOMPOK 4

DOSEN PENGAMPU: M. Harry Khomas Saputra

ARYA TIRTA (225443005)

MUHAMMAD DHAFIN RIZQULLAH (225443015)

NABILA KHALISHAH (225443021)

QURRATA AYUNI (225443023)

RIZAL AHMAD ALFAUZAN (225443026)

1AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2025/2026

PENDAHULUAN

1. Latar Belakang

Dalam dunia industri, mesin adalah aset paling penting yang harus dijaga. Salah satu musuh utama mesin adalah panas berlebih atau overheat. Jika suhu mesin tidak dipantau dengan baik, mesin bisa cepat rusak, produksi terhenti, dan biaya perbaikan pun menjadi sangat mahal.

Selama ini, pengecekan suhu terkadang masih dilakukan secara manual atau tidak terpantau setiap saat. Oleh karena itu, dibutuhkan sebuah sistem Dashboard Monitoring Suhu Mesin yang bisa menampilkan data secara real-time (langsung). Dengan dashboard ini, teknisi bisa memantau kondisi mesin dari mana saja tanpa harus mengecek satu per satu ke lapangan.

2. Masalah Yang ingin Diselesaikan

Kelompok kami fokus pada beberapa masalah utama, yaitu :

- Bagaimana cara memantau suhu mesin secara terus-menerus tanpa harus ada di lokasi?
- Bagaimana memberikan peringatan jika suhu mesin sudah melewati batas aman?
- Bagaimana menyajikan data suhu dalam bentuk visual (grafik) agar mudah dipahami oleh operator.

3. Tujuan Tugas

Tujuan dari analisis studi kasus ini adalah :

- Merancang sistem pemantauan suhu yang praktis dan akurat.
- Membuat tampilan dashboard yang memudahkan pengguna dalam membaca data suhu mesin.
- Mencegah kerusakan fatal pada mesin dengan deteksi dini melalui pemantauan suhu.

STUDI KASUS: DASHBOARD MONITORING SUHU MESIN

Membuat database studi_kasus yang berisi koleksi suhu_mesin, dengan sebuah skrip python bernama monitor.py

```
"""
monitor.py - Dashboard Monitoring Suhu Mesin
Studi Kasus Praktikum 6
"""

import os
import logging
from datetime import datetime, timedelta

import pandas as pd
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure,
ServerSelectionTimeoutError
from dotenv import load_dotenv

load_dotenv()

# ── Konfigurasi logging ke file alarm.log
──────────────────────────────────
logging.basicConfig(
    filename='alarm.log',
    level=logging.WARNING,
    format='%(asctime)s | %(levelname)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

SUHU_ALARM = 90  # derajat Celsius

def koneksi_db():
    """Buat koneksi ke MongoDB. Return (client, collection) atau raise
error."""
    client = MongoClient(os.getenv('MONGO_URI'),
serverSelectionTimeoutMS=3000)
    client.admin.command('ping')          # test koneksi
```

```python
    db = client['studi_kasus']
    return client, db['suhu_mesin']

def ambil_data_1jam(collection):
    """Ambil semua dokumen dalam 1 jam terakhir."""
    batas_waktu = datetime.utcnow() - timedelta(hours=1)
    return list(collection.find({'timestamp': {'$gt': batas_waktu}}))

def rata_rata_per_mesin(collection):
    """Hitung rata-rata suhu per mesin (1 jam terakhir) menggunakan
aggregation."""
    batas_waktu = datetime.utcnow() - timedelta(hours=1)
    pipeline = [
        {'$match': {'timestamp': {'$gt': batas_waktu}}},
        {'$group': {
            '_id': '$mesin',
            'rata_rata_suhu': {'$avg': '$suhu'},
            'jumlah_data': {'$sum': 1}
        }},
        {'$sort': {'_id': 1}}
    ]
    return list(collection.aggregate(pipeline))

def suhu_terkini_per_mesin(collection):
    """Ambil dokumen suhu terbaru untuk setiap mesin."""
    batas_waktu = datetime.utcnow() - timedelta(hours=1)
    pipeline = [
        {'$match': {'timestamp': {'$gt': batas_waktu}}},
        {'$sort': {'timestamp': -1}},
        {'$group': {
            '_id': '$mesin',
            'suhu_terkini': {'$first': '$suhu'},
            'timestamp': {'$first': '$timestamp'}
        }},
        {'$sort': {'_id': 1}}
    ]
    return list(collection.aggregate(pipeline))

def deteksi_alarm(collection):
```

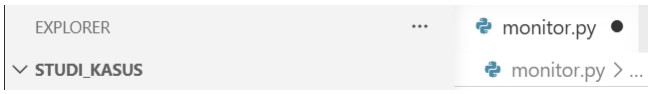
```
    """Deteksi mesin dengan suhu > 90 derajat dalam 1 jam terakhir."""
    batas_waktu = datetime.utcnow() - timedelta(hours=1)
    pipeline = [
        {'$match': {
            'timestamp': {'$gt': batas_waktu},
            'suhu': {'$gt': SUHU_ALARM}
        }},
        {'$group': {
            '_id': '$mesin',
            'suhu_max': {'$max': '$suhu'},
            'waktu_kejadian': {'$first': '$timestamp'}
        }},
        {'$sort': {'suhu_max': -1}}
    ]
    return list(collection.aggregate(pipeline))

def tampilkan_tabel(judul, df, lebar=60):
    """Tampilkan DataFrame sebagai tabel konsol sederhana."""
    print()
    print('=' * lebar)
    print(f'  {judul}')
    print('=' * lebar)
    if df.empty:
        print('  (Tidak ada data)')
    else:
        print(df.to_string(index=False))
    print('=' * lebar)

def catat_alarm(mesin, suhu_max, waktu):
    """Catat alarm ke file alarm.log."""
    logging.warning(f'ALARM | Mesin: {mesin} | Suhu Max:
{suhu_max:.1f}°C | Waktu: {waktu}')


def jalankan_dashboard():
    """Fungsi utama dashboard."""
    print('\n' + '╔' * 60)
    print('  DASHBOARD MONITORING SUHU MESIN')
```

```
    print(f'  Waktu laporan: {datetime.now().strftime("%Y-%m-%d
%H:%M:%S")}')
    print('═' * 60)

    try:
        client, collection = koneksi_db()
    except (ConnectionFailure, ServerSelectionTimeoutError):
        print('\n[OFFLINE] Sistem tidak dapat terhubung ke database
MongoDB.')
        print('          Periksa koneksi jaringan dan pastikan MongoDB
berjalan.')
        return
    except Exception as e:
        print(f'\n[ERROR] Koneksi gagal: {e}')
        return

    # ── 1. Suhu terkini per mesin
──────────────────────────────────────────
    terkini = suhu_terkini_per_mesin(collection)
    if terkini:
        df_terkini = pd.DataFrame(terkini).rename(columns={
            '_id': 'Mesin', 'suhu_terkini': 'Suhu Terkini (°C)',
'timestamp': 'Timestamp'
        })
        df_terkini['Timestamp'] =
pd.to_datetime(df_terkini['Timestamp']).dt.strftime('%H:%M:%S')
        tampilkan_tabel('SUHU TERKINI PER MESIN (1 JAM TERAKHIR)',
df_terkini)
    else:
        print('\n[INFO] Tidak ada data suhu dalam 1 jam terakhir.')

    # ── 2. Mesin alarm (suhu > 90°C)
────────────────────────────────────────
    alarm_list = deteksi_alarm(collection)
    if alarm_list:
        df_alarm = pd.DataFrame(alarm_list).rename(columns={
            '_id': 'Mesin', 'suhu_max': 'Suhu Max (°C)',
'waktu_kejadian': 'Waktu'
        })
```

```
        df_alarm['Waktu'] =
pd.to_datetime(df_alarm['Waktu']).dt.strftime('%H:%M:%S')
        tampilkan_tabel(f'⚠  ALARM: MESIN DENGAN SUHU > {SUHU_ALARM}°C',
df_alarm)

        # Catat ke alarm.log
        for row in alarm_list:
            catat_alarm(row['_id'], row['suhu_max'],
row['waktu_kejadian'])
        print(f'  ▸ {len(alarm_list)} alarm dicatat ke alarm.log')
    else:
        print('\n  ✔  Tidak ada mesin yang melewati batas suhu alarm.')

    # ── 3. Rata-rata suhu per mesin
──────────────────────────────────────
    rata = rata_rata_per_mesin(collection)
    if rata:
        df_rata = pd.DataFrame(rata).rename(columns={
            '_id': 'Mesin', 'rata_rata_suhu': 'Rata-rata Suhu (°C)',
'jumlah_data': 'Jumlah Data'
        })
        df_rata['Rata-rata Suhu (°C)'] = df_rata['Rata-rata Suhu
(°C)'].round(2)
        tampilkan_tabel('RATA-RATA SUHU PER MESIN (1 JAM TERAKHIR)',
df_rata)

    client.close()
    print('\n[INFO] Dashboard selesai diperbarui.\n')

if __name__ == '__main__':
    jalankan_dashboard()
```

Dokumentasi cara menjalankan:

1. Letakkan file monitor.py kedalam folder studi kasus



Di terminal, lakukan:

2. Buat virtual environtment

```
python -m venv venv

# Aktifkan (Windows):
venv\Scripts\activate
```

```
(venv) PS D:\piton mongo\studi_kasus>
```

3. Install Library

```
pip install pymongo pandas python-dotenv
```

```
Successfully installed dnspython-2.8.0 numpy-2.4.4 pandas-3.0.2 pymongo-4.17.0 py
thon-dateutil-2.9.0.post0 python-dotenv-1.2.2 six-1.17.0 tzdata-2026.2
```

4. Buat file .env
   New-Item .env
   Lalu isi ini:

```
MONGO_URI=mongodb://localhost:27017
```

5. Isi data dummy untuk testing
   Buat file dummy.py lalu isi dengan kode ini:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
```

```
import random

client = MongoClient('mongodb://localhost:27017')
col = client['studi_kasus']['suhu_mesin']

mesin_list = ['CNC-01', 'CNC-02', 'LATHE-01', 'MILL-01']
docs = []
for i in range(40):
    docs.append({
        'mesin': random.choice(mesin_list),
        'suhu': round(random.uniform(70, 100), 1),
        'timestamp': datetime.utcnow() -
timedelta(minutes=random.randint(0, 55))
    })

col.insert_many(docs)
print(f'{len(docs)} dokumen berhasil dimasukkan.')
```

6. Jalankan dummy.py
   Di terminal, ketik:

```
python dummy.py
```

```
40 dokumen berhasil dimasukkan.
```

Dokumen dummy berhasil dimasukkan.

7. Jalankan monitor.py
   Di terminal, ketik:

```
python monitor.py
```

```
  DASHBOARD MONITORING SUHU MESIN
  Waktu laporan: 2026-05-06 13:48:02

==============================================================
  SUHU TERKINI PER MESIN (1 JAM TERAKHIR)
==============================================================
   Mesin  Suhu Terkini (°C) Timestamp
  CNC-01               97.6  06:41:49
  CNC-02               83.2  06:40:49
LATHE-01               89.1  06:43:49
 MILL-01               74.5  06:34:49
==============================================================

==============================================================
  ⚠ ALARM: MESIN DENGAN SUHU > 90°C
==============================================================
   Mesin  Suhu Max (°C)    Waktu
  CNC-02           99.2 06:36:49
 MILL-01           99.1 06:06:49
  CNC-01           97.6 06:29:49
LATHE-01           93.2 05:59:49
==============================================================
  ▶ 4 alarm dicatat ke alarm.log

==============================================================
  RATA-RATA SUHU PER MESIN (1 JAM TERAKHIR)
==============================================================
   Mesin  Rata-rata Suhu (°C)  Jumlah Data
  CNC-01                85.09            7
  CNC-02                86.08           17
LATHE-01                80.79            7
 MILL-01                81.83            9
==============================================================

[INFO] Dashboard selesai diperbarui.
```

8. Cek file alarm.log

File `alarm.log` terbentuk otomatis karena di kode `monitor.py` ada baris ini:

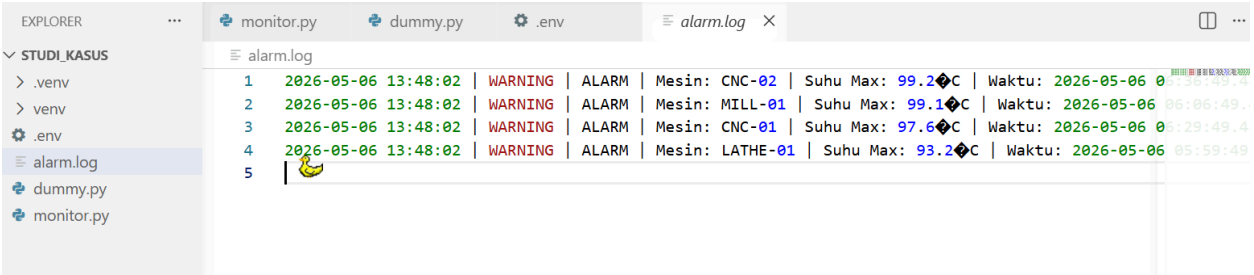
```
logging.basicConfig(
    filename='alarm.log',
    level=logging.WARNING,
    ...
)
```

Begitu `monitor.py` dijalankan, Python langsung membuat file `alarm.log` di folder yang sama, meskipun belum ada alarm yang tercatat sekalipun. Ini perilaku default dari modul `logging` Python.

File `alarm.log` baru akan terisi isinya kalau ada mesin yang suhunya di atas 90°C, lewat fungsi ini:

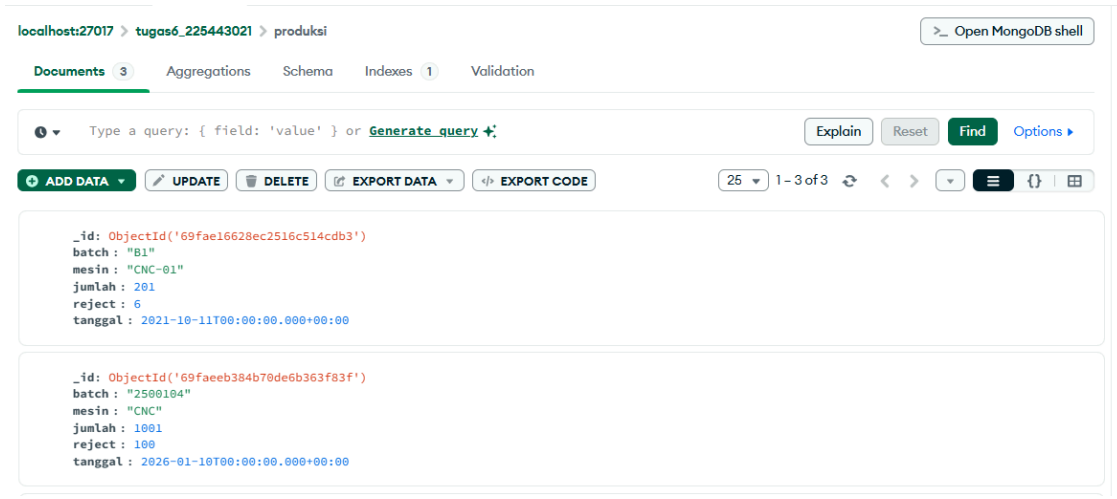
```
def catat_alarm(mesin, suhu_max, waktu):
    logging.warning(f'ALARM | Mesin: {mesin} | Suhu Max: {suhu_max:.1f}°C | Waktu: {waktu}')
```

Isi file alarm.log:



TUGAS TERSTRUKTUR (TUGAS MANDIRI)

1. Buat database baru bernama tugas6_NIM (masukan NIM) dengan koleksi produksi. Di dalam koleksi ini memiliki field batch (string), mesin (string), jumlah (int), tanggal (datetime).



2. Buat sebuah skrip python app_produksi.py yang berjalan dalam mode interaktif berbasis menu teks, menu yang harus tersedia:

   1. Input data produksi baru: pengguna memasukkan batch, mesin, jumlah, reject, dan tanggal melalui keyboard. Data disimpan ke MongoDB.

   2. Tampilkan data produksi per mesin: pengguna memasukkan nama mesin, lalu sistem menampilkan semua data produksi untuk mesin tersebut dalam bentuk tabel (bisa print DataFrame atau format manual).

   3. Hitung reject rate: sistem menjalankan agregasi untuk menghitung reject rate (reject/jumlah*100) per batch, lalu hanya menampilkan batch yang reject rate-nya di atas 5%.

   4. Ekspor laporan bulanan: pengguna memasukkan bulan dan tahun (format MM-YYYY). Sistem menghitung total jumlah dan total reject per mesin untuk bulan tersebut menggunakan agregasi, lalu menyimpan hasilnya ke file CSV dengan nama laporan_MM-YYYY.csv.

   5. Keluar.

```
import os
import logging
import pandas as pd
from tabulate import tabulate
```

```
from datetime import datetime
from pymongo import MongoClient
from dotenv import load_dotenv

# LOAD KONFIGURASI DARI .env
load_dotenv()

logging.basicConfig(
	filename='app.log',
	level=logging.INFO,
	format='%(asctime)s - %(levelname)s - %(message)s'
)

# KONEKSI KE MONGODB
try:
	# Mengambil connection string dari environment variable
	client = MongoClient(os.getenv("MONGO_URI"))
	db = client["tugas6_225443021"]
	col = db["produksi"]
	logging.info("Aplikasi dimulai - Koneksi sukses.")
except Exception as e:
	logging.error(f"Gagal koneksi database: {e}")
	print("Database tidak terdeteksi, pastikan MongoDB sudah jalan!")

# bagian menu

def input_data():
	"""Menu 1: Input data produksi baru"""
	try:
	print("\n--- Input Data Produksi ---")
```

```
        batch = input("Batch: ")
        mesin = input("Nama Mesin: ")
        jumlah = int(input("Jumlah Produksi: "))
        reject = int(input("Jumlah Reject: "))
        tgl_input = input("Tanggal (YYYY-MM-DD): ")
        tanggal = datetime.strptime(tgl_input, "%Y-%m-%d")

        payload = {
        "batch": batch,
        "mesin": mesin,
        "jumlah": jumlah,
        "reject": reject,
        "tanggal": tanggal
        }
        col.insert_one(payload)
        print(">> Data berhasil disimpan!")
        logging.info(f"INSERT: Batch {batch} ditambahkan.")
        except Exception as e:
        print(f">> Gagal input: {e}")
        logging.error(f"ERR_INPUT: {e}")

def tampil_mesin():
        """Menu 2: Tampilkan data per mesin dalam tabel"""
        nama_mesin = input("\nMasukkan nama mesin: ")
        query = {"mesin": nama_mesin}
        data = list(col.find(query))

    if data:
        table_data = []
        for d in data:
```

```
        table_data.append([
            d.get('batch'),
            d.get('mesin'),
            d.get('jumlah'),
            d.get('reject'),
        d.get('tanggal').strftime('%Y-%m-%d')
        ])

        headers = ["BATCH", "MESIN", "JUMLAH", "REJECT", "TANGGAL"]

        # Tampilkan pakai tabulate
        print("\n" + tabulate(table_data, headers=headers, tablefmt="fancy_grid"))
        logging.info(f"QUERY: Menampilkan tabel mesin {nama_mesin}")
        else:
        print("Data tidak ditemukan.")

def hitung_reject():
        """Menu 3: Agregasi reject rate > 5%"""
        pipeline = [
        {
        "$project": {
        "batch": 1,
        "reject_rate": {
            "$multiply": [{"$divide": ["$reject", "$jumlah"]}, 100]
        }
        }
        },
        {"$match": {"reject_rate": {"$gt": 5}}}
        ]
        hasil = list(col.aggregate(pipeline))
```

```
        print("\n--- Batch dengan Reject Rate > 5% ---")
        if hasil:
        for item in hasil:
        print(f"Batch: {item['batch']} | Rate: {item['reject_rate']:.2f}%")
        logging.info("AGGREGATE: Hitung reject rate sukses.")
        else:
        print("Aman! Tidak ada reject rate di atas 5%.")

def ekspor_csv():
        """Menu 4: Ekspor laporan bulanan"""
        print("\nFormat MM-YYYY (Contoh: 05-2026)")
        periode = input("Masukkan Bulan & Tahun: ")
        try:
        bln, thn = map(int, periode.split('-'))

        pipeline = [
        {
        "$match": {
                "tanggal": {
                "$gte": datetime(thn, bln, 1),
                "$lt": datetime(thn, bln + 1, 1) if bln < 12 else datetime(thn + 1, 1, 1)
                }
        }
        },
        {
        "$group": {
                  "_id": "$mesin",
                "total_jumlah": {"$sum": "$jumlah"},
                "total_reject": {"$sum": "$reject"}
```

```
        }
        }
        ]

        hasil = list(col.aggregate(pipeline))
        if hasil:
        df = pd.DataFrame(hasil).rename(columns={"_id": "mesin"})
        nama_file = f"laporan_{periode}.csv"
        df.to_csv(nama_file, index=False)
        print(f">> Berhasil diekspor ke {nama_file}")
        logging.info(f"EXPORT: Laporan {nama_file} dibuat.")
        else:
        print("Tidak ada data untuk periode tersebut.")
        except Exception as e:
        print("Kesalahan format.")
        logging.error(f"ERR_EXPORT: {e}")

# --- MENU UTAMA ---
def main():
        while True:
        print("\n=== APLIKASI DATA PRODUKSI ===")
        print("1. Input Data Baru")
        print("2. Tampilkan per Mesin")
        print("3. Cek Reject Rate (>5%)")
        print("4. Ekspor ke CSV")
        print("5. Keluar")

        pilih = input("Pilih (1-5): ")

        if pilih == '1': input_data()
```

```
        elif pilih == '2': tampil_mesin()
        elif pilih == '3': hitung_reject()
        elif pilih == '4': ekspor_csv()
        elif pilih == '5':
        print("Terimakasih!")
        break
        else:
        print("Pilihan salah!")

if __name__ == "__main__":
        main()
```

Output:

- Input data baru

```
=== APLIKASI DATA PRODUKSI ===
1. Input Data Baru
2. Tampilkan per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor ke CSV
5. Keluar
Pilih (1-5): 1

--- Input Data Produksi ---
Batch: B3
Nama Mesin: Drill
Jumlah Produksi: 2109
Jumlah Reject: 70
Tanggal (YYYY-MM-DD): 2026-01-09
>> Data berhasil disimpan!
```

- Tampilkan per mesin

```
=== APLIKASI DATA PRODUKSI ===
1. Input Data Baru
2. Tampilkan per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor ke CSV
5. Keluar
Pilih (1-5): 2

Masukkan nama mesin: Drill
```

BATCH	MESIN	JUMLAH	REJECT	TANGGAL
B3	Drill	2109	70	2026-01-09

- Cek reject rate

```
=== APLIKASI DATA PRODUKSI ===
1. Input Data Baru
2. Tampilkan per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor ke CSV
5. Keluar
Pilih (1-5): 3

--- Batch dengan Reject Rate > 5% ---
Batch: 2500104 | Rate: 9.99%
Batch: B1 | Rate: 9.09%
```

- Ekspor ke CSV

```
=== APLIKASI DATA PRODUKSI ===
1. Input Data Baru
2. Tampilkan per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor ke CSV
5. Keluar
Pilih (1-5): 4

Format MM-YYYY (Contoh: 05-2026)
Masukkan Bulan & Tahun: 01-2026
>> Berhasil diekspor ke laporan_01-2026.csv
```

laporan_01-2026.csv

```
mesin,total_jumlah,total_reject
Drill,2109,70
CNC,1001,100
```

- Keluar

```
=== APLIKASI DATA PRODUKSI ===
1. Input Data Baru
2. Tampilkan per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor ke CSV
5. Keluar
Pilih (1-5): 5
Terimakasih!
```

3. Menggunakan enviroment variable untuk connection string

```python
import os
import logging
import pandas as pd
from tabulate import tabulate
from datetime import datetime
from pymongo import MongoClient
from dotenv import load_dotenv

# LOAD KONFIGURASI DARI .env
load_dotenv()

logging.basicConfig(
    filename='app.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# KONEKSI KE MONGODB
try:
    # Mengambil connection string dari environment variable
    client = MongoClient(os.getenv("MONGO_URI"))
    db = client["tugas6_225443021"]
    col = db["produksi"]
```

4. Implementasikan logging ke file app.log pada setiap operasi besar (insert, query, eeror) wajib tercatat dengan timestamp

```python
logging.basicConfig(
    filename='app.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# KONEKSI KE MONGODB
try:
    # Mengambil connection string dari environment variable
    client = MongoClient(os.getenv("MONGO_URI"))
    db = client["tugas6_225443021"]
    col = db["produksi"]
    logging.info("Aplikasi dimulai - Koneksi sukses.")
except Exception as e:
    logging.error(f"Gagal koneksi database: {e}")
    print("Database tidak terdeteksi, pastikan MongoDB sudah jalan!")
```

5. Struktur yang modular
   - Input data

```python
def input_data():
    """Menu 1: Input data produksi baru"""
    try:
        print("\n--- Input Data Produksi ---")
        batch = input("Batch: ")
        mesin = input("Nama Mesin: ")
        jumlah = int(input("Jumlah Produksi: "))
        reject = int(input("Jumlah Reject: "))
        tgl_input = input("Tanggal (YYYY-MM-DD): ")
        tanggal = datetime.strptime(tgl_input, "%Y-%m-%d")

        payload = {
            "batch": batch,
            "mesin": mesin,
            "jumlah": jumlah,
            "reject": reject,
            "tanggal": tanggal
        }
        col.insert_one(payload)
        print(">> Data berhasil disimpan!")
        logging.info(f"INSERT: Batch {batch} ditambahkan.")
    except Exception as e:
        print(f">> Gagal input: {e}")
        logging.error(f"ERR_INPUT: {e}")
```

   - Tampilan mesin

```python
def tampil_mesin():
    """Menu 2: Tampilkan data per mesin dalam tabel"""
    nama_mesin = input("\nMasukkan nama mesin: ")
    query = {"mesin": nama_mesin}
    data = list(col.find(query))

    if data:
        table_data = []
        for d in data:
                table_data.append([
                    d.get('batch'),
                    d.get('mesin'),
                    d.get('jumlah'),
                    d.get('reject'),
                    d.get('tanggal').strftime('%Y-%m-%d')
                ])

        headers = ["BATCH", "MESIN", "JUMLAH", "REJECT", "TANGGAL"]

        # Tampilkan pakai tabulate
        print("\n" + tabulate(table_data, headers=headers, tablefmt="fancy_grid"))
        logging.info(f"QUERY: Menampilkan tabel mesin {nama_mesin}")
    else:
        print("Data tidak ditemukan.")
```

- Hitung reject

```python
def hitung_reject():
    """Menu 3: Agregasi reject rate > 5%"""
    pipeline = [
        {
            "$project": {
                "batch": 1,
                "reject_rate": {
                    "$multiply": [{"$divide": ["$reject", "$jumlah"]}, 100]
                }
            }
        },
        {"$match": {"reject_rate": {"$gt": 5}}}
    ]
    hasil = list(col.aggregate(pipeline))

    print("\n--- Batch dengan Reject Rate > 5% ---")
    if hasil:
        for item in hasil:
            print(f"Batch: {item['batch']} | Rate: {item['reject_rate']:.2f}%")
        logging.info("AGGREGATE: Hitung reject rate sukses.")
    else:
        print("Aman! Tidak ada reject rate di atas 5%.")
```

- Ekspor CSV

```python
def ekspor_csv():
    """Menu 4: Ekspor laporan bulanan"""
    print("\nFormat MM-YYYY (Contoh: 05-2026)")
    periode = input("Masukkan Bulan & Tahun: ")
    try:
        bln, thn = map(int, periode.split('-'))

        pipeline = [
            {
                "$match": {
                    "tanggal": {
                        "$gte": datetime(thn, bln, 1),
                        "$lt": datetime(thn, bln + 1, 1) if bln < 12 else datetime
                        (thn + 1, 1, 1)
                    }
                }
            },
            {
                "$group": {
                    "_id": "$mesin",
                    "total_jumlah": {"$sum": "$jumlah"},
                    "total_reject": {"$sum": "$reject"}
                }
            }
        ]

        hasil = list(col.aggregate(pipeline))
        if hasil:
            df = pd.DataFrame(hasil).rename(columns={"_id": "mesin"})
            nama_file = f"laporan_{periode}.csv"
            df.to_csv(nama_file, index=False)
            print(f">> Berhasil diekspor ke {nama_file}")
            logging.info(f"EXPORT: Laporan {nama_file} dibuat.")
        else:
            print("Tidak ada data untuk periode tersebut.")
    except Exception as e:
        print("Kesalahan format.")
        logging.error(f"ERR_EXPORT: {e}")
```

6. Dokumentasi singkat cara menjalankan program di file [README.md]

```
# DOKUMENTASI PROGRAM - SISTEM DATA PRODUKSI


Tugas ini dibangun untuk memenuhi kriteria Tugas Terstruktur (Tugas Mandiri) pembangunan
aplikasi input data produksi dan laporan berbasis MongoDB.


# 1. Persiapan Lingkungan

- Pastikan MongoDB Community Server sudah terinstal dan berjalan (Running).

- Gunakan Python 3.x.

- Instalasi library yang dibutuhkan:

  pip install pymongo pandas python-dotenv tabulate


# 2. Konfigurasi Database (.env)

Aplikasi menggunakan environment variable untuk koneksi database. Pastikan file `.env`
tersedia di root folder dengan isi:

MONGO_URI=mongodb://localhost:27017/


#3. Cara Menjalankan Program

1. Buka Terminal/PowerShell di folder project.

2. Aktifkan Virtual Environment:

   .\venv\Scripts\activate

3. Jalankan aplikasi:

   python app_produksi.py


# 4. Fitur Aplikasi

- Menu 1: Input data produksi (tersimpan ke koleksi 'produksi').

- Menu 2: Tampilkan data per mesin dalam format tabel rapi.

- Menu 3: Agregasi reject rate untuk mencari batch dengan reject > 5%.

- Menu 4: Ekspor laporan total produksi & reject per mesin ke file CSV.

- Logging: Semua aktivitas operasional (insert, query, error) otomatis tercatat di file `app.log`.
```

5. Struktur Folder

- app_produksi.py : Skrip utama aplikasi (Modular).
- .env : Konfigurasi connection string.
- app.log : File catatan aktivitas (Logging).
- README.md : Panduan penggunaan program.

TUGAS KELOMPOK

Pembagian Peran dan Tugas Setiap Anggota:

Anggota 1 – MQTT Publisher Simulasi

- Anggota 1 bertanggung jawab membuat file pub_udara.py yang berfungsi sebagai MQTT Publisher untuk mensimulasikan data sensor secara realtime. Program ini menghasilkan data dari beberapa sensor pada area cold storage dengan informasi seperti sensor_id, lokasi, tipe_sensor, nilai sensor, dan timestamp. Data yang dihasilkan kemudian dikirim ke topik MQTT pabrik/udara setiap 10 detik menggunakan broker MQTT. Simulasi data dibuat menyerupai kondisi nyata dengan rentang nilai normal, namun terdapat probabilitas sekitar 5% untuk menghasilkan kondisi anomali seperti nilai PM2.5 tinggi, suhu tidak normal, atau kadar amonia berlebih guna mensimulasikan kondisi berbahaya pada sistem monitoring.

Anggota 2 – MQTT Subscriber dan Penyimpanan Data

- Anggota 2 bertugas membuat file sub_udara.py yang berfungsi sebagai MQTT Subscriber sekaligus penghubung antara MQTT dan MongoDB. Program ini menerima data dari topik pabrik/udara, melakukan parsing data JSON, kemudian menyimpannya ke database MongoDB kelompok6 pada koleksi kualitas_udara. Selain itu, sistem juga melakukan deteksi anomali secara otomatis dengan memeriksa nilai PM2.5. Jika nilai PM2.5 melebihi batas aman (>150), maka program akan membuat dokumen peringatan pada koleksi alert yang berisi informasi sensor, lokasi, nilai PM2.5, timestamp, dan pesan peringatan “PM2.5 berbahaya!”. Program juga dilengkapi error handling untuk menangani kesalahan koneksi MQTT, MongoDB, maupun parsing data.

Anggota 3 – Analisis Data dan Dashboard

- Anggota 3 bertanggung jawab membuat file analisis_dashboard.py yang digunakan untuk melakukan analisis data historis secara on-demand. Program ini mengambil data 24 jam terakhir dari koleksi kualitas_udara, kemudian mengolahnya menggunakan MongoDB Aggregation Pipeline seperti $match, $group, $avg, dan $sort untuk menghitung rata-rata data sensor per jam berdasarkan lokasi. Hasil analisis divisualisasikan dalam bentuk grafik menggunakan library matplotlib sehingga memudahkan pemantauan kondisi cold storage. Dashboard yang dihasilkan disimpan dalam file dashboard_udara.png, sedangkan data mentah 24 jam terakhir juga diekspor ke file data_24jam.csv untuk kebutuhan dokumentasi dan analisis lanjutan.

Anggota 4 – Status Sensor Terkini

- Anggota 4 bertugas membuat file status_udara.py yang digunakan untuk menampilkan status terkini dari setiap sensor. Program bekerja dengan melakukan query ke MongoDB untuk mengambil data terbaru berdasarkan sensor_id, kemudian menentukan status sensor menjadi “Normal”, “Waspada”, atau “Bahaya” berdasarkan nilai PM2.5 yang diterima. Hasil monitoring ditampilkan secara realtime di terminal dan diperbarui secara berkala agar pengguna dapat memantau kondisi sensor dengan mudah. Program ini juga menampilkan

ringkasan jumlah sensor dalam kondisi normal maupun bahaya sehingga mempermudah proses monitoring sistem secara keseluruhan.

Anggota 5 – Dokumentasi dan Laporan

- Anggota 5 bertanggung jawab dalam penyusunan dokumentasi dan laporan akhir proyek. Tugas yang dilakukan meliputi pembuatan file README.md yang berisi penjelasan sistem dan langkah-langkah menjalankan program secara lengkap, mulai dari setup database hingga menjalankan MQTT Publisher dan Subscriber. Selain itu, anggota 5 juga menyusun laporan kelompok dalam format PDF yang mencakup flowchart sistem, pembagian tugas anggota, penjelasan setiap file program, tangkapan layar hasil pengujian, serta analisis hasil implementasi sistem monitoring IoT. Anggota 5 juga membantu menyiapkan bahan presentasi kelompok untuk menjelaskan alur kerja sistem dan hasil proyek pada saat presentasi praktikum.

Langkah-langkah pengerjaan:

1. Buat folder baru
2. Buka foler tersebut di vscode
3. Buat file dengan nama setup_mongodb.py dan isi dengan kode berikut

```
"""
setup_mongodb.py - Inisialisasi Database & Isi Sample Data
Menghubungkan ke MongoDB, membuat database + koleksi,
membuat index, dan mengisi data contoh realistis.

Jalankan SEKALI sebelum menjalankan sistem:
    python setup_mongodb.py
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')

import os
import random
from datetime import datetime, timezone, timedelta

from pymongo import MongoClient, ASCENDING, DESCENDING, errors

# ─── Konfigurasi ──────────────────────────────────────────────────────
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
```

```python
DB_NAME   = "kelompok6"

SENSORS = [
    {"sensor_id": "TEMP-A1",  "lokasi": "Cold Room A", "tipe": "suhu"},
    {"sensor_id": "TEMP-B1",  "lokasi": "Cold Room B", "tipe": "suhu"},
    {"sensor_id": "HUM-A1",   "lokasi": "Cold Room A", "tipe":
"kelembapan"},
    {"sensor_id": "HUM-B1",   "lokasi": "Cold Room B", "tipe":
"kelembapan"},
    {"sensor_id": "NH3-A1",   "lokasi": "Cold Room A", "tipe":
"amonia"},
    {"sensor_id": "NH3-B1",   "lokasi": "Cold Room B", "tipe":
"amonia"},
    {"sensor_id": "DOOR-A1",  "lokasi": "Cold Room A", "tipe": "pintu"},
    {"sensor_id": "DOOR-B1",  "lokasi": "Cold Room B", "tipe": "pintu"},
    {"sensor_id": "PRESS-A1", "lokasi": "Cold Room A", "tipe":
"tekanan"},
    {"sensor_id": "PRESS-B1", "lokasi": "Cold Room B", "tipe":
"tekanan"},
]


def generate_sample(sensor: dict, ts: datetime, anomaly: bool = False)
-> dict:
    """Hasilkan satu dokumen sensor realistis."""
    tipe = sensor["tipe"]
    base = {
        "sensor_id":   sensor["sensor_id"],
        "lokasi":      sensor["lokasi"],
        "tipe_sensor": tipe,
        "timestamp":   ts.isoformat(),
        "suhu":        None,
        "kelembapan":  None,
        "amonia":      None,
        "status_pintu": None,
        "tekanan":     None,
        "pm25":        None,
    }
```

```
    if tipe == "suhu":
        base["suhu"] = round(random.uniform(-8.0, -5.0) if anomaly else
random.uniform(-22.0, -18.0), 2)
        base["pm25"] = round(random.uniform(160, 200) if anomaly else
random.uniform(5, 40), 2)

    elif tipe == "kelembapan":
        base["kelembapan"] = round(random.uniform(55, 65) if anomaly
else random.uniform(85, 95), 2)
        base["pm25"]       = round(random.uniform(160, 200) if anomaly
else random.uniform(5, 40), 2)

    elif tipe == "amonia":
        nh3 = round(random.uniform(30, 50) if anomaly else
random.uniform(0.5, 9.5), 2)
        base["amonia"] = nh3
        base["pm25"]   = round(nh3 * 6 if anomaly else nh3 * 1.5, 2)

    elif tipe == "pintu":
        base["status_pintu"] = 1 if anomaly else 0
        base["pm25"] = round(random.uniform(165, 200) if anomaly else
random.uniform(5, 20), 2)

    elif tipe == "tekanan":
        base["tekanan"] = round(random.uniform(5, 18) if anomaly else
random.uniform(50, 80), 2)
        base["pm25"]    = round(random.uniform(158, 195) if anomaly else
random.uniform(10, 30), 2)

    return base


def seed_data(col_data, col_alert):
    """Masukkan data historis 24 jam ke MongoDB."""
    now    = datetime.now(timezone.utc)
    cutoff = now - timedelta(hours=24)

    docs        = []
    alert_docs  = []
```

```
    ts          = cutoff

    print("[2] Mengisi data historis (setiap 10 menit selama 24
jam)...")
    while ts <= now:
        for sensor in SENSORS:
            anomaly = random.random() < 0.05
            doc     = generate_sample(sensor, ts, anomaly)
            docs.append(doc)

            # Buat alert jika anomali
            if anomaly and (doc.get("pm25") or 0) > 150:
                tipe = sensor["tipe"]
                if tipe == "amonia":
                    pesan = f"Amonia berbahaya! NH₃={doc['amonia']} ppm"
                elif tipe == "pintu":
                    pesan = "Pintu TERBUKA terlalu lama!"
                elif tipe == "tekanan":
                    pesan = f"Tekanan rendah: {doc['tekanan']} Pa
(penumpukan es)"
                else:
                    pesan = f"PM2.5 berbahaya! Nilai={doc['pm25']}"

                alert_docs.append({
                    "sensor_id":   sensor["sensor_id"],
                    "lokasi":      sensor["lokasi"],
                    "tipe_sensor": tipe,
                    "pm25":        doc["pm25"],
                    "timestamp":   ts.isoformat(),
                    "pesan":       pesan,
                    "created_at":  ts.isoformat(),
                })

        ts += timedelta(minutes=10)

    col_data.insert_many(docs)
    print(f"    → {len(docs)} dokumen ke 'kualitas_udara'")

    if alert_docs:
```

```
        col_alert.insert_many(alert_docs)
        print(f"    → {len(alert_docs)} alert ke 'alert'")
    else:
        print("    → 0 alert (tidak ada anomali)")


def main():
    print(f"\n{'='*60}")
    print("   SETUP DATABASE MONGODB - Cold Storage IoT")
    print(f"{'='*60}\n")

    # 1. Koneksi
    try:
        client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
        client.server_info()
        print(f"[OK] Terhubung ke MongoDB: {MONGO_URI}")
    except errors.ServerSelectionTimeoutError as e:
        print(f"[No] Gagal konek MongoDB: {e}")
        return

    db        = client[DB_NAME]
    col_data  = db["kualitas_udara"]
    col_alert = db["alert"]

    print(f"[1] Database: {DB_NAME}")

    # 2. Drop & recreate collections (fresh start)
    col_data.drop()
    col_alert.drop()
    print("    → Koleksi lama dihapus (reset)")

    # 3. Buat index
    col_data.create_index([("sensor_id", ASCENDING), ("timestamp",
DESCENDING)])
    col_data.create_index([("timestamp", DESCENDING)])
    col_data.create_index([("tipe_sensor", ASCENDING)])
    col_alert.create_index([("sensor_id", ASCENDING), ("timestamp",
DESCENDING)])
    print("    → Index dibuat")
```

```
    # 4. Seed data
    seed_data(col_data, col_alert)

    # 5. Verifikasi
    print(f"\n[3] Verifikasi:")
    print(f"    kualitas_udara : {col_data.count_documents({}):,}
dokumen")
    print(f"    alert          : {col_alert.count_documents({}):,}
dokumen")
    print(f"\n[✓] Setup selesai! Database '{DB_NAME}' siap digunakan.")

    # Tampilkan contoh dokumen
    print("\n[4] Contoh dokumen 'kualitas_udara':")
    sample = col_data.find_one({}, {"_id": 0})
    if sample:
        for k, v in sample.items():
            if v is not None:
                print(f"    {k:15}: {v}")

    print("\n[5] Contoh dokumen 'alert':")
    sample_alert = col_alert.find_one({}, {"_id": 0})
    if sample_alert:
        for k, v in sample_alert.items():
            if v is not None:
                print(f"    {k:15}: {v}")

    client.close()


if __name__ == "__main__":
    main()
```

4. Buat file api_server.py dan masukkan kode berikut:

```
"""
api_server.py - REST API Backend untuk Web Monitoring Cold Storage
Menyediakan endpoint untuk dashboard, status sensor, alert, dan export
CSV.

Jalankan: python api_server.py
Buka browser: http://localhost:5000
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')

import csv
import io
import os
from datetime import datetime, timezone, timedelta

from flask import Flask, jsonify, send_file, request
from flask_cors import CORS
from pymongo import MongoClient, errors, DESCENDING

# ─── Konfigurasi
──────────────────────────────────────────────────────────────────────
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME   = "kelompok6"
COL_DATA  = "kualitas_udara"
COL_ALERT = "alert"

app = Flask(__name__, static_folder=".", static_url_path="")
CORS(app)

SENSOR_IDS = [
    "TEMP-A1", "TEMP-B1",
    "HUM-A1",  "HUM-B1",
    "NH3-A1",  "NH3-B1",
    "DOOR-A1", "DOOR-B1",
    "PRESS-A1","PRESS-B1",
]
```

```
# ─── Koneksi MongoDB ──────────────────────────────────────────────────────
def get_db():
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    return client[DB_NAME], client


# ─── Endpoint: Status terkini semua sensor ────────────────────────────────
@app.route("/api/status")
def api_status():
    try:
        db, client = get_db()
        col = db[COL_DATA]
        col_alert = db[COL_ALERT]

        results = []
        for sid in SENSOR_IDS:
            doc = col.find_one({"sensor_id": sid}, sort=[("timestamp",
DESCENDING)])
            if doc:
                doc.pop("_id", None)
                results.append(doc)
            else:
                results.append({
                    "sensor_id": sid,
                    "lokasi": "—",
                    "tipe_sensor": "—",
                    "pm25": None,
                    "timestamp": None,
                })

        total_alerts = col_alert.count_documents({})
        client.close()

        return jsonify({
            "sensors": results,
            "total_alerts": total_alerts,
```

```
            "updated_at": datetime.now(timezone.utc).isoformat(),
        })
    except errors.ServerSelectionTimeoutError as e:
        return jsonify({"error": f"MongoDB tidak terjangkau:
{str(e)}"}), 503


# ─── Endpoint: Data grafik 24 jam (agregasi per jam)
────────────────────────
@app.route("/api/chart")
def api_chart():
    try:
        db, client = get_db()
        col = db[COL_DATA]
        cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()

        sensor_types = [
            {"tipe": "suhu",       "field": "suhu"},
            {"tipe": "kelembapan", "field": "kelembapan"},
            {"tipe": "amonia",     "field": "amonia"},
            {"tipe": "tekanan",    "field": "tekanan"},
        ]

        chart_data = {}
        for cfg in sensor_types:
            pipeline = [
                {"$match": {
                    "timestamp":   {"$gte": cutoff},
                    "tipe_sensor": cfg["tipe"],
                    cfg["field"]:  {"$ne": None}
                }},
                {"$addFields": {
                    "ts_date": {"$dateFromString": {"dateString":
"$timestamp"}}
                }},
                {"$group": {
                    "_id": {
                        "lokasi": "$lokasi",
```

```
                        "hour":   {"$hour": "$ts_date"}
                    },
                    "avg_value": {"$avg": f"${cfg['field']}"}
                }},
                {"$sort": {"_id.hour": 1}}
            ]
            results = list(col.aggregate(pipeline))

            lokasi_data = {}
            for doc in results:
                loc = doc["_id"]["lokasi"]
                hour = doc["_id"]["hour"]
                if loc not in lokasi_data:
                    lokasi_data[loc] = []
                lokasi_data[loc].append({
                    "hour": hour,
                    "value": round(doc["avg_value"], 2)
                })

            chart_data[cfg["tipe"]] = lokasi_data

        client.close()
        return jsonify(chart_data)
    except errors.ServerSelectionTimeoutError as e:
        return jsonify({"error": str(e)}), 503


# ─── Endpoint: Alert terbaru
──────────────────────────────────────────────────────
@app.route("/api/alerts")
def api_alerts():
    try:
        limit = int(request.args.get("limit", 20))
        db, client = get_db()
        col_alert = db[COL_ALERT]

        docs = list(col_alert.find(
            {}, {"_id": 0}
        ).sort("timestamp", DESCENDING).limit(limit))
```

```
        client.close()
        return jsonify({"alerts": docs, "total": len(docs)})
    except errors.ServerSelectionTimeoutError as e:
        return jsonify({"error": str(e)}), 503


# ── Endpoint: Ringkasan statistik
──────────────────────────────────────────
@app.route("/api/summary")
def api_summary():
    try:
        db, client = get_db()
        col = db[COL_DATA]
        col_alert = db[COL_ALERT]
        cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()

        total_docs = col.count_documents({"timestamp": {"$gte":
cutoff}})
        total_alerts = col_alert.count_documents({"timestamp": {"$gte":
cutoff}})
        total_danger = col.count_documents({
            "timestamp": {"$gte": cutoff},
            "pm25": {"$gt": 150}
        })

        # Rata-rata suhu 24 jam terakhir
        pipeline_suhu = [
            {"$match": {"tipe_sensor": "suhu", "timestamp": {"$gte":
cutoff}, "suhu": {"$ne": None}}},
            {"$group": {"_id": None, "avg": {"$avg": "$suhu"}}}
        ]
        r_suhu = list(col.aggregate(pipeline_suhu))
        avg_suhu = round(r_suhu[0]["avg"], 2) if r_suhu else None

        # Rata-rata kelembapan
        pipeline_hum = [
            {"$match": {"tipe_sensor": "kelembapan", "timestamp":
```

```
{"$gte": cutoff}, "kelembapan": {"$ne": None}}},
            {"$group": {"_id": None, "avg": {"$avg": "$kelembapan"}}}
        ]
        r_hum = list(col.aggregate(pipeline_hum))
        avg_hum = round(r_hum[0]["avg"], 2) if r_hum else None

        client.close()
        return jsonify({
            "total_data_24h": total_docs,
            "total_alerts_24h": total_alerts,
            "total_danger": total_danger,
            "avg_suhu": avg_suhu,
            "avg_kelembapan": avg_hum,
        })
    except errors.ServerSelectionTimeoutError as e:
        return jsonify({"error": str(e)}), 503


# ── Endpoint: Export CSV 24 jam terakhir
──────────────────────────────────────
@app.route("/api/export/csv")
def api_export_csv():
    try:
        db, client = get_db()
        col = db[COL_DATA]
        cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()

        docs = list(col.find(
            {"timestamp": {"$gte": cutoff}},
            {"_id": 0}
        ).sort("timestamp", 1))

        client.close()

        fieldnames = [
            "sensor_id", "lokasi", "tipe_sensor", "timestamp",
            "suhu", "kelembapan", "amonia", "status_pintu", "tekanan",
"pm25"
```

```
        ]

        output = io.StringIO()
        writer = csv.DictWriter(output, fieldnames=fieldnames,
extrasaction="ignore")
        writer.writeheader()
        writer.writerows(docs)
        output.seek(0)

        filename =
f"data_udara_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
        return send_file(
            io.BytesIO(output.getvalue().encode("utf-8")),
            mimetype="text/csv",
            as_attachment=True,
            download_name=filename
        )
    except errors.ServerSelectionTimeoutError as e:
        return jsonify({"error": str(e)}), 503


# ─── Serve index.html
──────────────────────────────────────────────────────────────
@app.route("/")
def index():
    return send_file("index.html")


if __name__ == "__main__":
    print("\n" + "="*60)
    print("   COLD STORAGE WEB MONITORING API")
    print("="*60)
    print(f"MongoDB  : {MONGO_URI} → {DB_NAME}")
    print(f"Server   : http://localhost:5000")
    print(f"Dashboard: http://localhost:5000/dashboard.html")
    print("="*60 + "\n")
    app.run(host="0.0.0.0", port=5000, debug=True)
```

5. Buat file pub_udara.py dan masukkan kode berikut:

```
"""
pub_udara.py - MQTT Publisher Simulasi Sensor Cold Storage
Anggota 1: MQTT Publisher

Mensimulasikan 5 jenis sensor untuk sistem monitoring Cold Storage
(ruang penyimpanan daging).
Setiap 10 detik, data dikirim ke topik MQTT 'pabrik/udara'.
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')

import json
import random
import time
from datetime import datetime, timezone

import paho.mqtt.client as mqtt

# ─── Konfigurasi MQTT ──────────────────────────────────────────────────────
BROKER_HOST = "broker.hivemq.com"
BROKER_PORT = 1883
TOPIC       = "pabrik/udara"
INTERVAL    = 10  # detik antar pengiriman

# ─── Definisi Sensor (5 jenis x 2 lokasi = 10 sensor) ──────────────────────
SENSORS = [
    {"sensor_id": "TEMP-A1",  "lokasi": "Cold Room A", "tipe": "suhu"},
    {"sensor_id": "TEMP-B1",  "lokasi": "Cold Room B", "tipe": "suhu"},
    {"sensor_id": "HUM-A1",   "lokasi": "Cold Room A", "tipe":
"kelembapan"},
    {"sensor_id": "HUM-B1",   "lokasi": "Cold Room B", "tipe":
"kelembapan"},
    {"sensor_id": "NH3-A1",   "lokasi": "Cold Room A", "tipe":
"amonia"},
```

```
    {"sensor_id": "NH3-B1",   "lokasi": "Cold Room B", "tipe":
"amonia"},
    {"sensor_id": "DOOR-A1",  "lokasi": "Cold Room A", "tipe": "pintu"},
    {"sensor_id": "DOOR-B1",  "lokasi": "Cold Room B", "tipe": "pintu"},
    {"sensor_id": "PRESS-A1", "lokasi": "Cold Room A", "tipe":
"tekanan"},
    {"sensor_id": "PRESS-B1", "lokasi": "Cold Room B", "tipe":
"tekanan"},
]


def generate_sensor_data(sensor: dict) -> dict:
    """
    Membuat data sensor simulasi berdasarkan tipe sensor.
    Probabilitas 5% untuk kondisi anomali/berbahaya.
    """
    tipe = sensor["tipe"]
    is_anomaly = random.random() < 0.05  # 5% peluang anomali

    if tipe == "suhu":
        # Normal: -18°C s/d -22°C | Anomali: di atas -10°C (pintu
terbuka/kerusakan)
        suhu = round(random.uniform(-8.0, -5.0), 2) if is_anomaly else
round(random.uniform(-22.0, -18.0), 2)
        payload = {
            "suhu": suhu,
            "pm25": None, "kelembapan": None, "amonia": None,
            "status_pintu": None, "tekanan": None
        }

    elif tipe == "kelembapan":
        # Normal: 85%-95% | Anomali: di bawah 70% (risiko freezer burn)
        kelembapan = round(random.uniform(50.0, 65.0), 2) if is_anomaly
else round(random.uniform(85.0, 95.0), 2)
        payload = {
            "kelembapan": kelembapan,
            "pm25": None, "suhu": None, "amonia": None,
            "status_pintu": None, "tekanan": None
        }
```

```
    elif tipe == "amonia":
        # Normal: 0-10 ppm | Anomali: >25 ppm (berbahaya - kebocoran
refrigran)
        amonia = round(random.uniform(28.0, 50.0), 2) if is_anomaly else
round(random.uniform(0.5, 9.5), 2)
        # pm25 digunakan sebagai proxy anomali (> 150 untuk trigger
alert)
        pm25_equiv = amonia * 6.0 if is_anomaly else amonia * 1.5
        payload = {
            "amonia": amonia,
            "pm25": round(pm25_equiv, 2),
            "suhu": None, "kelembapan": None,
            "status_pintu": None, "tekanan": None
        }

    elif tipe == "pintu":
        # Normal: 0 (tertutup) | Anomali: 1 (terbuka)
        status_pintu = 1 if is_anomaly else 0
        # pm25 tinggi jika pintu terbuka (proxy anomali)
        pm25_equiv = round(random.uniform(160.0, 200.0), 2) if
is_anomaly else round(random.uniform(5.0, 20.0), 2)
        payload = {
            "status_pintu": status_pintu,
            "pm25": pm25_equiv,
            "suhu": None, "kelembapan": None,
            "amonia": None, "tekanan": None
        }

    elif tipe == "tekanan":
        # Normal: 50-80 Pa | Anomali: < 20 Pa (penumpukan es/frost)
        tekanan = round(random.uniform(5.0, 18.0), 2) if is_anomaly else
round(random.uniform(50.0, 80.0), 2)
        pm25_equiv = round(random.uniform(155.0, 190.0), 2) if
is_anomaly else round(random.uniform(10.0, 30.0), 2)
        payload = {
            "tekanan": tekanan,
            "pm25": pm25_equiv,
            "suhu": None, "kelembapan": None,
```

```
            "amonia": None, "status_pintu": None
        }
    else:
        payload = {}

    # Gabungkan data umum
    data = {
        "sensor_id":    sensor["sensor_id"],
        "lokasi":       sensor["lokasi"],
        "tipe_sensor":  tipe,
        "timestamp":    datetime.now(timezone.utc).isoformat(),
        **payload
    }
    return data


def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print(f"[✓] Terhubung ke broker MQTT
{BROKER_HOST}:{BROKER_PORT}")
    else:
        print(f"[✗] Gagal terhubung, kode: {rc}")


def on_publish(client, userdata, mid):
    print(f"    → Pesan #{mid} terkirim")


def main():
    client = mqtt.Client(client_id="publisher-cold-storage")
    client.on_connect = on_connect
    client.on_publish  = on_publish

    try:
        client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
        client.loop_start()
    except ConnectionRefusedError:
        print("[✗] Tidak dapat terhubung ke broker. Pastikan Mosquitto
berjalan.")
```

```
        return

    print(f"\n{'='*60}")
    print("   COLD STORAGE SENSOR PUBLISHER")
    print(f"{'='*60}")
    print(f"Topik  : {TOPIC}")
    print(f"Interval: setiap {INTERVAL} detik")
    print(f"Sensor  : {len(SENSORS)} unit\n")

    cycle = 1
    try:
        while True:
            print(f"\n[Siklus #{cycle}]
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
            for sensor in SENSORS:
                data    = generate_sensor_data(sensor)
                payload = json.dumps(data, ensure_ascii=False)
                result  = client.publish(TOPIC, payload, qos=1)
                status  = "ANOMALI ⚠️" if (data.get("pm25") or 0) > 150
else "normal"
                print(f"  [{sensor['sensor_id']:10s}]
{sensor['lokasi']:12s} | {sensor['tipe']:10s} | {status}")
            cycle += 1
            time.sleep(INTERVAL)
    except KeyboardInterrupt:
        print("\n[i] Publisher dihentikan oleh pengguna.")
    finally:
        client.loop_stop()
        client.disconnect()


if __name__ == "__main__":
    main()
```

Output:

```
[✓] Terhubung ke broker MQTT broker.hivemq.com:1883
    → Pesan #231 terkirim
    → Pesan #232 terkirim
    → Pesan #233 terkirim
    → Pesan #234 terkirim
    → Pesan #235 terkirim
    → Pesan #236 terkirim
    → Pesan #237 terkirim
    → Pesan #238 terkirim
    → Pesan #239 terkirim
    → Pesan #240 terkirim
[✓] Terhubung ke broker MQTT broker.hivemq.com:1883
```

6. Buat file sub_udara.py dan isi dengan kode berikut:

```
"""
sub_udara.py - MQTT Subscriber & Penyimpanan ke MongoDB
Anggota 2: MQTT Subscriber

Menerima pesan dari topik 'pabrik/udara', menyimpan ke MongoDB,
dan mendeteksi anomali secara real-time.

Database : kelompok6
Koleksi  : kualitas_udara  → semua data sensor
           alert           → dokumen peringatan anomali
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')

import json
import os
from datetime import datetime, timezone

import paho.mqtt.client as mqtt
```

```
from pymongo import MongoClient, errors

# ─── Konfigurasi ──────────────────────────────────────────────────────────
MQTT_HOST   = "broker.hivemq.com"
MQTT_PORT   = 1883
MQTT_TOPIC  = "pabrik/udara"

MONGO_URI   = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME     = "kelompok6"
COL_DATA    = "kualitas_udara"
COL_ALERT   = "alert"

PM25_DANGER_THRESHOLD = 150  # µg/m³ (atau ekuivalen)

# ─── Inisialisasi MongoDB ─────────────────────────────────────────────────
try:
    mongo_client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    mongo_client.server_info()  # test koneksi
    db         = mongo_client[DB_NAME]
    col_data   = db[COL_DATA]
    col_alert  = db[COL_ALERT]
    print(f"[✓] MongoDB terhubung: {MONGO_URI} → {DB_NAME}")
except errors.ServerSelectionTimeoutError as e:
    print(f"[✗] MongoDB tidak dapat dijangkau: {e}")
    raise SystemExit(1)


def handle_anomaly(data: dict):
    """Menyimpan dokumen alert dan mencetak notifikasi mencolok."""
    pm25 = data.get("pm25") or 0

    # Tentukan pesan berdasarkan tipe sensor
    tipe = data.get("tipe_sensor", "")
    if tipe == "amonia":
        pesan = f"Amonia berbahaya! NH₃={data.get('amonia')} ppm (proxy
PM2.5={pm25})"
    elif tipe == "pintu":
```

```
        pesan = f"Pintu TERBUKA terlalu lama!
Status={data.get('status_pintu')}"
    elif tipe == "tekanan":
        pesan = f"Tekanan diferensial rendah! Kemungkinan penumpukan es.
Tekanan={data.get('tekanan')} Pa"
    else:
        pesan = f"PM2.5 berbahaya! Nilai={pm25} µg/m³"

    alert_doc = {
        "sensor_id": data.get("sensor_id"),
        "lokasi":    data.get("lokasi"),
        "tipe_sensor": data.get("tipe_sensor"),
        "pm25":      pm25,
        "timestamp": data.get("timestamp"),
        "pesan":     pesan,
        "created_at": datetime.now(timezone.utc).isoformat(),
    }

    try:
        col_alert.insert_one(alert_doc)
    except errors.PyMongoError as e:
        print(f"[✗] Gagal menyimpan alert: {e}")
        return

    # Notifikasi mencolok di konsol
    border = "🔴" * 30
    print(f"\n{border}")
    print(f"  ⚠️  PERINGATAN ANOMALI TERDETEKSI ⚠️")
    print(f"  Sensor  : {alert_doc['sensor_id']}
({alert_doc['lokasi']})")
    print(f"  Tipe    : {alert_doc['tipe_sensor'].upper()}")
    print(f"  Pesan   : {alert_doc['pesan']}")
    print(f"  Waktu   : {alert_doc['timestamp']}")
    print(f"{border}\n")


def on_message(client, userdata, msg):
    """Callback saat pesan MQTT diterima."""
    try:
```

```
        payload = msg.payload.decode("utf-8")
        data    = json.loads(payload)
    except (UnicodeDecodeError, json.JSONDecodeError) as e:
        print(f"[✗] Gagal parse pesan: {e}")
        return

    # Simpan ke koleksi utama
    try:
        col_data.insert_one(data)
        pm25 = data.get("pm25") or 0
        status = "⚠️ ANOMALI" if pm25 > PM25_DANGER_THRESHOLD else "✅
Normal"
        print(f"[{data.get('sensor_id'):10s}]
{data.get('tipe_sensor'):10s} | PM25≈{pm25:6.1f} | {status}")
    except errors.PyMongoError as e:
        print(f"[✗] Gagal menyimpan data: {e}")
        return

    # Deteksi anomali
    pm25 = data.get("pm25") or 0
    if pm25 > PM25_DANGER_THRESHOLD:
        handle_anomaly(data)


def on_connect(client, userdata, flags, rc):
    if rc == 0:
        client.subscribe(MQTT_TOPIC, qos=1)
        print(f"[✓] Terhubung ke MQTT & subscribe topik
'{MQTT_TOPIC}'")
    else:
        print(f"[✗] Gagal terhubung MQTT, kode: {rc}")


def on_disconnect(client, userdata, rc):
    if rc != 0:
        print(f"[!] Terputus dari broker (kode {rc}), mencoba
reconnect...")
```

```python
def main():
    client = mqtt.Client(client_id="subscriber-cold-storage")
    client.on_connect    = on_connect
    client.on_message    = on_message
    client.on_disconnect = on_disconnect

    print(f"\n{'='*60}")
    print("   COLD STORAGE SENSOR SUBSCRIBER")
    print(f"{'='*60}")
    print(f"Broker : {MQTT_HOST}:{MQTT_PORT}")
    print(f"Topik  : {MQTT_TOPIC}")
    print(f"MongoDB: {DB_NAME} | {COL_DATA} | {COL_ALERT}\n")

    try:
        client.connect(MQTT_HOST, MQTT_PORT, keepalive=60)
        client.loop_forever()  # blokir & auto-reconnect
    except KeyboardInterrupt:
        print("\n[i] Subscriber dihentikan.")
    except ConnectionRefusedError:
        print("[✗] Tidak dapat terhubung ke broker MQTT.")
    finally:
        client.disconnect()
        mongo_client.close()


if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\mdhaf\Downloads\kelompok6> python sub_udara.py

============================================================
   COLD STORAGE SENSOR SUBSCRIBER
============================================================
Broker : broker.hivemq.com:1883
Topik  : pabrik/udara
MongoDB: kelompok6 | kualitas_udara | alert

[√] Terhubung ke MQTT & subscribe topik 'pabrik/udara'
[!] Terputus dari broker (kode 7), mencoba reconnect...
[√] Terhubung ke MQTT & subscribe topik 'pabrik/udara'
[!] Terputus dari broker (kode 7), mencoba reconnect...
[√] Terhubung ke MQTT & subscribe topik 'pabrik/udara'
[!] Terputus dari broker (kode 7), mencoba reconnect...
[√] Terhubung ke MQTT & subscribe topik 'pabrik/udara'
[TEMP-A1   ] suhu       | PM25≈   0.0 | ✅ Normal
[TEMP-B1   ] suhu       | PM25≈   0.0 | ✅ Normal
[HUM-A1    ] kelembapan | PM25≈   0.0 | ✅ Normal
[HUM-B1    ] kelembapan | PM25≈   0.0 | ✅ Normal
[NH3-A1    ] amonia     | PM25≈  13.1 | ✅ Normal
[NH3-B1    ] amonia     | PM25≈   1.5 | ✅ Normal
[DOOR-A1   ] pintu      | PM25≈   5.2 | ✅ Normal
[DOOR-B1   ] pintu      | PM25≈  16.2 | ✅ Normal
[PRESS-A1  ] tekanan    | PM25≈  26.8 | ✅ Normal
[PRESS-B1  ] tekanan    | PM25≈  26.8 | ✅ Normal
[TEMP-A1   ] suhu       | PM25≈   0.0 | ✅ Normal
```

7. Buat file analisis_dashboard.py dan isi dengan kode berikut:

```
"""
analisis_dashboard.py - Analisis Data & Visualisasi Dashboard
Anggota 3: Analisis dan Dashboard

Mengambil data 24 jam terakhir dari MongoDB, membuat grafik perbandingan
antar lokasi, dan mengekspor ke CSV.

Output:
  - dashboard_udara.png  : grafik tren sensor per lokasi
  - data_24jam.csv       : data mentah 24 jam terakhir
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')
```

```python
import os
import csv
from datetime import datetime, timezone, timedelta

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from pymongo import MongoClient, errors

# ─── Konfigurasi ──────────────────────────────────────────────────────────
MONGO_URI  = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME    = "kelompok6"
COL_DATA   = "kualitas_udara"

OUTPUT_PNG = "dashboard_udara.png"
OUTPUT_CSV = "data_24jam.csv"


def get_mongo_collection():
    """Koneksi ke MongoDB dan kembalikan koleksi."""
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    client.server_info()
    return client[DB_NAME][COL_DATA], client


def fetch_last_24h(col) -> list:
    """
    Mengambil semua dokumen dalam 24 jam terakhir.

    Returns:
        list[dict]: daftar dokumen sensor
    """
    cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()
    cursor = col.find(
        {"timestamp": {"$gte": cutoff}},
```

```
        {"_id": 0}
    ).sort("timestamp", 1)
    return list(cursor)


def aggregate_hourly(col, tipe_sensor: str, field: str) -> dict:
    """
    Agregasi rata-rata nilai per jam per lokasi untuk tipe sensor
tertentu.

    Args:
        col         : koleksi MongoDB
        tipe_sensor : tipe sensor (misal 'suhu', 'kelembapan')
        field       : nama field yang dirata-rata (misal 'suhu',
'kelembapan')

    Returns:
        dict: {lokasi: [(jam, rata_rata), ...]}
    """
    cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()
    pipeline = [
        {"$match": {
            "timestamp":   {"$gte": cutoff},
            "tipe_sensor": tipe_sensor,
            field:         {"$ne": None}
        }},
        {"$addFields": {
            "ts_date": {"$dateFromString": {"dateString": "$timestamp"}}
        }},
        {"$group": {
            "_id": {
                "lokasi": "$lokasi",
                "hour":   {"$hour": "$ts_date"}
            },
            "avg_value": {"$avg": f"${field}"}
        }},
        {"$sort": {"_id.hour": 1}}
    ]
```

```
    results = list(col.aggregate(pipeline))

    lokasi_data: dict = {}
    for doc in results:
        lokasi = doc["_id"]["hour"]  # nanti kita pakai hour sebagai x
        loc    = doc["_id"]["lokasi"]
        if loc not in lokasi_data:
            lokasi_data[loc] = []
        lokasi_data[loc].append((doc["_id"]["hour"],
round(doc["avg_value"], 2)))

    return lokasi_data


def export_csv(data: list, filepath: str):
    """Ekspor data ke file CSV."""
    if not data:
        print("[!] Tidak ada data untuk diekspor.")
        return

    fieldnames = [
        "sensor_id", "lokasi", "tipe_sensor", "timestamp",
        "suhu", "kelembapan", "amonia", "status_pintu", "tekanan",
"pm25"
    ]
    with open(filepath, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames,
extrasaction="ignore")
        writer.writeheader()
        writer.writerows(data)

    print(f"[✓] Data diekspor ke '{filepath}' ({len(data)} baris)")


def plot_dashboard(col):
    """
    Membuat grafik dashboard multi-panel untuk setiap tipe sensor.
    Grafik disimpan sebagai dashboard_udara.png.
    """
```

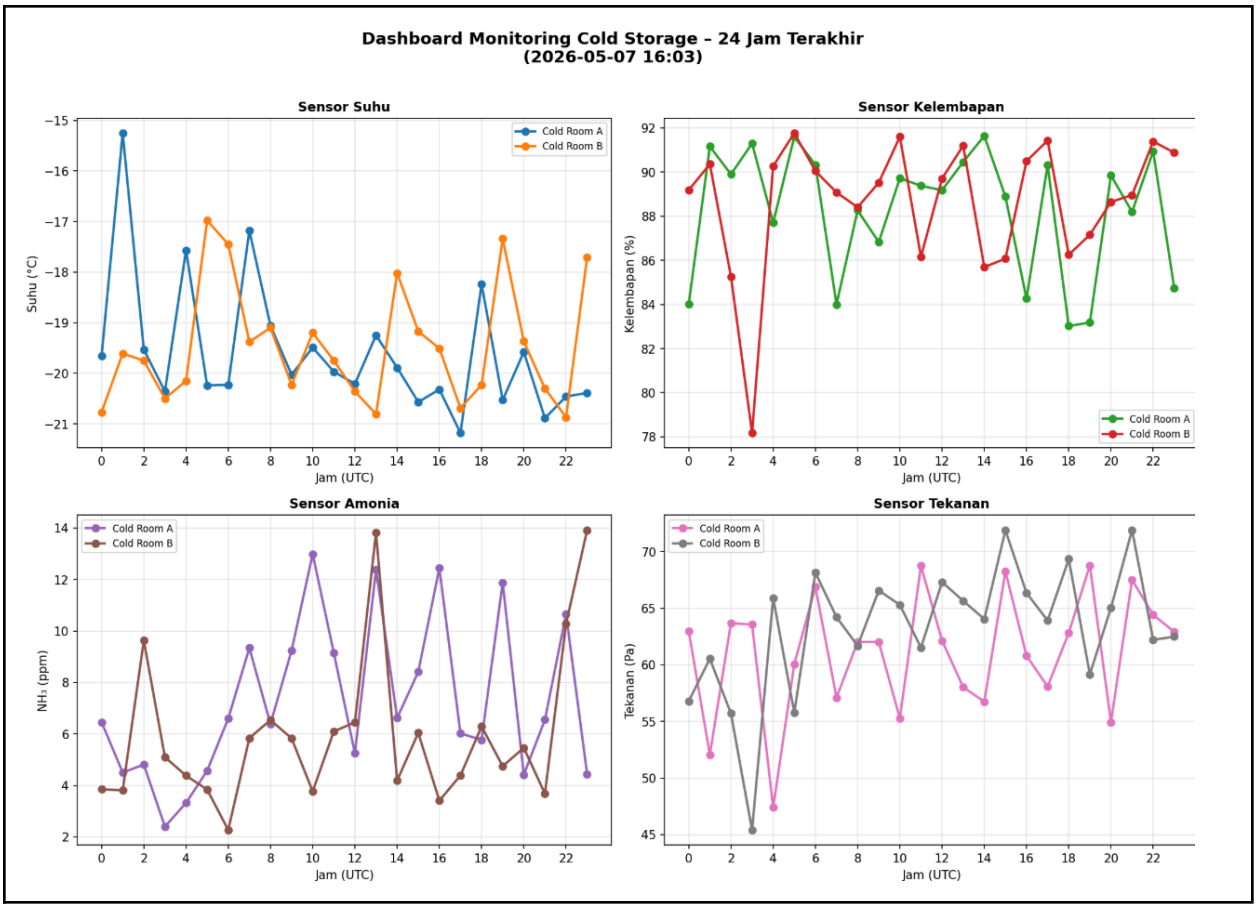
```
    configs = [
        {"tipe": "suhu",      "field": "suhu",       "ylabel": "Suhu
(°C)",    "color": ["#1f77b4", "#ff7f0e"]},
        {"tipe": "kelembapan","field": "kelembapan",  "ylabel":
"Kelembapan (%)", "color": ["#2ca02c", "#d62728"]},
        {"tipe": "amonia",    "field": "amonia",      "ylabel": "NH₃
(ppm)",    "color": ["#9467bd", "#8c564b"]},
        {"tipe": "tekanan",   "field": "tekanan",     "ylabel": "Tekanan
(Pa)", "color": ["#e377c2", "#7f7f7f"]},
    ]

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle(
        f"Dashboard Monitoring Cold Storage - 24 Jam Terakhir\n"
        f"({datetime.now().strftime('%Y-%m-%d %H:%M')})",
        fontsize=14, fontweight="bold", y=1.01
    )
    axes_flat = axes.flatten()

    for i, cfg in enumerate(configs):
        ax  = axes_flat[i]
        data = aggregate_hourly(col, cfg["tipe"], cfg["field"])

        if not data:
            ax.text(0.5, 0.5, "Belum ada data", ha="center",
va="center",
                    transform=ax.transAxes, fontsize=11, color="gray")
            ax.set_title(f"Sensor {cfg['tipe'].title()}")
            continue

        for j, (lokasi, points) in enumerate(sorted(data.items())):
            hours  = [p[0] for p in points]
            values = [p[1] for p in points]
            color  = cfg["color"][j % len(cfg["color"])]
            ax.plot(hours, values, marker="o", label=lokasi,
color=color, linewidth=2)

        ax.set_title(f"Sensor {cfg['tipe'].title()}", fontsize=11,
fontweight="bold")
```

```
        ax.set_xlabel("Jam (UTC)")
        ax.set_ylabel(cfg["ylabel"])
        ax.legend(fontsize=8)
        ax.grid(True, alpha=0.3)
        ax.set_xticks(range(0, 24, 2))

    # Panel ke-5: status pintu (bar chart)
    door_data = aggregate_hourly(col, "pintu", "pm25")
    ax5 = axes_flat[3] if len(configs) < 4 else None
    # (sudah 4 panel, cukup)

    plt.tight_layout()
    plt.savefig(OUTPUT_PNG, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[✓] Grafik disimpan ke '{OUTPUT_PNG}'")


def main():
    print(f"\n{'='*60}")
    print("   COLD STORAGE - ANALISIS & DASHBOARD")
    print(f"{'='*60}\n")

    try:
        col, client = get_mongo_collection()
    except errors.ServerSelectionTimeoutError as e:
        print(f"[✗] MongoDB tidak dapat dijangkau: {e}")
        return

    # 1. Ambil data 24 jam terakhir
    print("[1] Mengambil data 24 jam terakhir...")
    data = fetch_last_24h(col)
    print(f"    → {len(data)} dokumen ditemukan")

    # 2. Ekspor ke CSV
    print("[2] Mengekspor ke CSV...")
    export_csv(data, OUTPUT_CSV)

    # 3. Plot dashboard
    print("[3] Membuat grafik dashboard...")
```

```python
    plot_dashboard(col)

    print(f"\n[✓] Selesai! Output: {OUTPUT_PNG}, {OUTPUT_CSV}")
    client.close()


if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\mdhaf\Downloads\kelompok6> python analisis_dashboard.py

  ============================================================
     COLD STORAGE – ANALISIS & DASHBOARD
  ============================================================

  [1] Mengambil data 24 jam terakhir...
      → 21 dokumen ditemukan
  [2] Mengekspor ke CSV...
  [✓] Data diekspor ke 'data_24jam.csv' (21 baris)
  [3] Membuat grafik dashboard...
  [✓] Grafik disimpan ke 'dashboard_udara.png'

  [✓] Selesai! Output: dashboard_udara.png, data_24jam.csv
PS C:\Users\mdhaf\Downloads\kelompok6>
```



8. Buat file status_udara.py dan isi dengan kode berikut:

```
"""
status_udara.py - Status Terkini Setiap Sensor Cold Storage
Anggota 4: Status Terkini & Dokumentasi

Pendekatan 2: Query MongoDB untuk dokumen terbaru setiap sensor,
lalu tampilkan statusnya di konsol dalam tabel yang rapi.

Status:
  ✅ Normal  - kondisi optimal
  ⚠️ Waspada - perlu perhatian (pm25 50-150)
  🔴 Bahaya  - tindakan segera diperlukan (pm25 > 150)
"""
import sys
sys.stdout.reconfigure(encoding='utf-8')
```

```python
import os
import time
from datetime import datetime

from pymongo import MongoClient, errors, DESCENDING

# ─── Konfigurasi ──────────────────────────────────────────────────────────
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME   = "kelompok6"
COL_DATA  = "kualitas_udara"
COL_ALERT = "alert"

SENSOR_IDS = [
    "TEMP-A1", "TEMP-B1",
    "HUM-A1",  "HUM-B1",
    "NH3-A1",  "NH3-B1",
    "DOOR-A1", "DOOR-B1",
    "PRESS-A1","PRESS-B1",
]

REFRESH_INTERVAL = 10  # detik sebelum refresh tampilan


def get_status_label(pm25) -> tuple:
    """
    Kembalikan (emoji, label) berdasarkan nilai pm25.
    pm25 digunakan sebagai proxy universal untuk anomali.
    """
    if pm25 is None:
        return ("❓", "Tidak Diketahui")
    elif pm25 <= 50:
        return ("✅", "Normal")
    elif pm25 <= 150:
        return ("⚠️ ", "Waspada")
    else:
        return ("🔴", "Bahaya!")
```

```python
def get_sensor_value_str(doc: dict) -> str:
    """Format nilai utama sensor berdasarkan tipe."""
    tipe = doc.get("tipe_sensor", "")
    if tipe == "suhu":
        v = doc.get("suhu")
        return f"Suhu      : {v}°C" if v is not None else "-"
    elif tipe == "kelembapan":
        v = doc.get("kelembapan")
        return f"Kelembapan: {v}%" if v is not None else "-"
    elif tipe == "amonia":
        v = doc.get("amonia")
        return f"NH₃       : {v} ppm" if v is not None else "-"
    elif tipe == "pintu":
        v = doc.get("status_pintu")
        return f"Pintu     : {'TERBUKA ' if v == 1 else 'Tertutup'}"
    elif tipe == "tekanan":
        v = doc.get("tekanan")
        return f"Tekanan   : {v} Pa" if v is not None else "-"
    return "-"


def fetch_latest_status(col) -> list:
    """
    Query dokumen terbaru untuk setiap sensor_id.

    Returns:
        list[dict]: satu dokumen per sensor
    """
    results = []
    for sid in SENSOR_IDS:
        doc = col.find_one(
            {"sensor_id": sid},
            sort=[("timestamp", DESCENDING)]
        )
        if doc:
            results.append(doc)
        else:
            results.append({
```

```
                "sensor_id":   sid,
                "lokasi":      "—",
                "tipe_sensor": "—",
                "pm25":        None,
                "timestamp":   "Belum ada data",
            })
    return results


def count_alerts(col_alert) -> int:
    """Hitung jumlah alert dalam koleksi."""
    return col_alert.count_documents({})


def display_status(docs: list, total_alerts: int):
    """Cetak tabel status sensor ke konsol."""
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print("\033[2J\033[H", end="")  # clear screen

    print("=" * 68)
    print(f"   COLD STORAGE MONITORING — STATUS TERKINI")
    print(f"   Diperbarui: {now}  |  Total Alert: {total_alerts}")
    print("=" * 68)
    print(f"{'No':>3} {'Sensor ID':12} {'Lokasi':14} {'Tipe':10}
{'Status':20} {'Nilai'}")
    print("-" * 68)

    for i, doc in enumerate(docs, 1):
        pm25           = doc.get("pm25")
        emoji, label   = get_status_label(pm25)
        nilai          = get_sensor_value_str(doc)
        ts             = doc.get("timestamp", "")[:19] if
doc.get("timestamp") else "—"

        print(
            f"{i:>3} "
            f"{doc.get('sensor_id','—'):12} "
            f"{doc.get('lokasi','—'):14} "
            f"{doc.get('tipe_sensor','—'):10} "
```

```
            f"{emoji} {label:15} "
            f"{nilai}"
        )

    print("-" * 68)

    # Ringkasan
    n_bahaya  = sum(1 for d in docs if (d.get("pm25") or 0) > 150)
    n_waspada = sum(1 for d in docs if 50 < (d.get("pm25") or 0) <= 150)
    n_normal  = sum(1 for d in docs if (d.get("pm25") or 0) <= 50 and
d.get("pm25") is not None)
    print(f"\n  Ringkasan: 🔴 Bahaya={n_bahaya}  ⚠️  Waspada={n_waspada}
✅ Normal={n_normal}")
    print(f"  Tekan Ctrl+C untuk keluar. Refresh setiap
{REFRESH_INTERVAL} detik.\n")


def main():
    try:
        client    = MongoClient(MONGO_URI,
serverSelectionTimeoutMS=5000)
        client.server_info()
        db        = client[DB_NAME]
        col       = db[COL_DATA]
        col_alert = db[COL_ALERT]
        print(f"[✓] MongoDB terhubung: {DB_NAME}")
    except errors.ServerSelectionTimeoutError as e:
        print(f"[✗] MongoDB tidak terjangkau: {e}")
        return

    try:
        while True:
            docs         = fetch_latest_status(col)
            total_alerts = count_alerts(col_alert)
            display_status(docs, total_alerts)
            time.sleep(REFRESH_INTERVAL)
    except KeyboardInterrupt:
        print("\n[i] Status monitor dihentikan.")
    finally:
```

```
        client.close()


if __name__ == "__main__":
    main()
```

Output:

```
======================================================================
   COLD STORAGE MONITORING — STATUS TERKINI
   Diperbarui: 2026-05-07 08:23:18  |  Total Alert: 10
======================================================================
 No Sensor ID    Lokasi         Tipe       Status              Nilai
----------------------------------------------------------------------
  1 TEMP-A1      Cold Room A    suhu       ❓ Tidak Diketahui Suhu      : -18.62°C
  2 TEMP-B1      Cold Room B    suhu       ❓ Tidak Diketahui Suhu      : -18.38°C
  3 HUM-A1       Cold Room A    kelembapan ❓ Tidak Diketahui Kelembapan: 91.94%
  4 HUM-B1       Cold Room B    kelembapan ❓ Tidak Diketahui Kelembapan: 94.86%
  5 NH3-A1       Cold Room A    amonia     ✅ Normal          NH₃       : 1.78 ppm
  6 NH3-B1       Cold Room B    amonia     ✅ Normal          NH₃       : 6.64 ppm
  7 DOOR-A1      Cold Room A    pintu      ✅ Normal          Pintu     : Tertutup
  8 DOOR-B1      Cold Room B    pintu      ✅ Normal          Pintu     : Tertutup
  9 PRESS-A1     Cold Room A    tekanan    ✅ Normal          Tekanan   : 68.95 Pa
 10 PRESS-B1     Cold Room B    tekanan    ✅ Normal          Tekanan   : 76.08 Pa
----------------------------------------------------------------------

  Ringkasan: 🔴 Bahaya=0  ⚠️ Waspada=0  ✅ Normal=6
  Tekan Ctrl+C untuk keluar. Refresh setiap 10 detik.
```

9. Buat file index.html dan isi dengan kode berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Cold Storage Monitoring — Kelompok 6</title>
<link
href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;7
00&family=Syne:wght@400;600;800&display=swap" rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
<style>
  :root {
    --bg:        #05080f;
    --surface:   #0c1220;
    --surface2:  #111927;
    --border:    #1e2d42;
    --accent:    #00e5ff;
    --accent2:   #ff6b35;
    --green:     #00e676;
    --yellow:    #ffd740;
    --red:       #ff1744;
    --text:      #cdd6f4;
    --muted:     #4a5568;
    --font-head: 'Syne', sans-serif;
    --font-mono: 'JetBrains Mono', monospace;
  }

  *, *::before, *::after { box-sizing: border-box; margin: 0; padding:
0; }

  body {
    background: var(--bg);
    color: var(--text);
    font-family: var(--font-mono);
    min-height: 100vh;
```

```
  overflow-x: hidden;
}

/* Grid scanline bg */
body::before {
  content: '';
  position: fixed; inset: 0;
  background-image:
    linear-gradient(rgba(0,229,255,.025) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,229,255,.025) 1px, transparent 1px);
  background-size: 40px 40px;
  pointer-events: none; z-index: 0;
}

/* ─── Header ──────────────────────────────────────── */
header {
  position: relative; z-index: 10;
  display: flex; align-items: center; justify-content: space-between;
  padding: 18px 32px;
  border-bottom: 1px solid var(--border);
  background: rgba(12,18,32,.95);
  backdrop-filter: blur(10px);
}

.logo-area { display: flex; align-items: center; gap: 14px; }
.logo-icon {
  width: 44px; height: 44px;
  background: var(--accent);
  clip-path: polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);
  display: flex; align-items: center; justify-content: center;
  font-size: 20px; animation: pulse-icon 3s ease-in-out infinite;
}
@keyframes pulse-icon {
  0%,100% { box-shadow: 0 0 0 0 rgba(0,229,255,.4); }
  50%      { box-shadow: 0 0 0 12px rgba(0,229,255,0); }
}

h1 {
  font-family: var(--font-head);
```

```
    font-size: 1.25rem; font-weight: 800;
    letter-spacing: -.02em;
    color: #fff;
  }
  h1 span { color: var(--accent); }

  .header-meta { font-size: .7rem; color: var(--muted); margin-top: 2px;
}

  .refresh-status {
    display: flex; align-items: center; gap: 10px;
    font-size: .75rem; color: var(--muted);
  }

  .live-dot {
    width: 8px; height: 8px; border-radius: 50%;
    background: var(--green);
    animation: blink 1.2s ease-in-out infinite;
  }
  .live-dot.error { background: var(--red); animation: none; }
  @keyframes blink {
    0%,100% { opacity: 1; } 50% { opacity: .2; }
  }

  #countdown-bar {
    height: 2px; background: var(--border); position: relative;
    overflow: hidden;
  }
  #countdown-fill {
    height: 100%;
    background: linear-gradient(90deg, var(--accent), var(--accent2));
    transition: width .2s linear;
  }

  /* ─── Main layout ──────────────────────────────────── */
  main {
    position: relative; z-index: 5;
    padding: 24px 32px;
    display: flex; flex-direction: column; gap: 24px;
```

```
}

/* ── Stat cards ──────────────────────────────────── */
.stat-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 14px;
}

.stat-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 18px 20px;
  position: relative; overflow: hidden;
  transition: border-color .3s;
}
.stat-card::before {
  content: '';
  position: absolute; top: 0; left: 0; right: 0; height: 2px;
  background: var(--accent-card, var(--accent));
}
.stat-card:hover { border-color: var(--accent); }

.stat-label {
  font-size: .65rem; color: var(--muted); text-transform: uppercase;
  letter-spacing: .1em; margin-bottom: 8px;
}
.stat-value {
  font-family: var(--font-head);
  font-size: 2rem; font-weight: 800; color: #fff;
  line-height: 1;
}
.stat-sub { font-size: .65rem; color: var(--muted); margin-top: 6px; }

/* ── Section headers ─────────────────────────────── */
.section-head {
  display: flex; align-items: center; gap: 10px;
  font-family: var(--font-head);
```

```
    font-size: .85rem; font-weight: 600;
    color: var(--accent); text-transform: uppercase; letter-spacing:
.12em;
    margin-bottom: 14px;
  }
  .section-head::after {
    content: ''; flex: 1; height: 1px; background: var(--border);
  }

  /* ─── Sensor grid ──────────────────────────────────── */
  .sensor-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
    gap: 14px;
  }

  .sensor-card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 16px 18px;
    transition: all .3s;
    position: relative; overflow: hidden;
    animation: card-in .4s ease both;
  }
  @keyframes card-in {
    from { opacity: 0; transform: translateY(10px); }
    to   { opacity: 1; transform: translateY(0); }
  }
  .sensor-card::after {
    content: '';
    position: absolute; left: 0; top: 0; bottom: 0; width: 3px;
    background: var(--status-color, var(--muted));
    border-radius: 3px 0 0 3px;
    transition: background .3s;
  }

  .sensor-card.status-normal  { --status-color: var(--green); }
  .sensor-card.status-warning { --status-color: var(--yellow);
```

```
border-color: rgba(255,215,64,.3); }
  .sensor-card.status-danger  { --status-color: var(--red);
border-color: rgba(255,23,68,.3); animation: shake .3s ease, card-in .4s
ease both; }
  @keyframes shake {
    0%,100% { transform: translateX(0); }
    25%     { transform: translateX(-4px); }
    75%     { transform: translateX(4px); }
  }

  .sensor-top {
    display: flex; justify-content: space-between; align-items:
flex-start;
    margin-bottom: 12px;
  }
  .sensor-id {
    font-size: .8rem; font-weight: 700; color: var(--accent);
    letter-spacing: .05em;
  }
  .sensor-lokasi { font-size: .65rem; color: var(--muted); margin-top:
2px; }
  .sensor-badge {
    font-size: .6rem; padding: 3px 8px; border-radius: 20px;
    font-weight: 700; text-transform: uppercase; letter-spacing: .08em;
  }
  .badge-normal  { background: rgba(0,230,118,.15); color: var(--green);
}
  .badge-warning { background: rgba(255,215,64,.15); color:
var(--yellow); }
  .badge-danger  { background: rgba(255,23,68,.15); color: var(--red); }
  .badge-unknown { background: rgba(74,85,104,.2); color: var(--muted);
}

  .sensor-value {
    font-family: var(--font-head);
    font-size: 1.6rem; font-weight: 800; color: #fff;
    margin-bottom: 4px;
  }
  .sensor-tipe { font-size: .65rem; color: var(--muted); text-transform:
```

```
uppercase; }
  .sensor-ts   { font-size: .6rem; color: var(--muted); margin-top: 8px;
}

  /* ─── Charts ──────────────────────────────────────── */
  .chart-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(420px, 1fr));
    gap: 16px;
  }
  .chart-card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 20px 22px;
  }
  .chart-title {
    font-family: var(--font-head);
    font-size: .85rem; font-weight: 600; color: var(--text);
    margin-bottom: 16px;
  }
  .chart-wrap { position: relative; height: 200px; }

  /* ─── Alerts table ────────────────────────────────── */
  .alert-box {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 12px;
    overflow: hidden;
  }
  .alert-head {
    display: flex; align-items: center; justify-content: space-between;
    padding: 16px 20px;
    border-bottom: 1px solid var(--border);
  }
  .alert-list { max-height: 300px; overflow-y: auto; }
  .alert-list::-webkit-scrollbar { width: 4px; }
  .alert-list::-webkit-scrollbar-thumb { background: var(--border);
border-radius: 2px; }
```

```
  .alert-row {
    display: grid;
    grid-template-columns: 100px 120px 1fr 150px;
    gap: 12px; align-items: center;
    padding: 12px 20px;
    border-bottom: 1px solid rgba(30,45,66,.5);
    font-size: .72rem;
    transition: background .2s;
  }
  .alert-row:hover { background: var(--surface2); }
  .alert-row:last-child { border-bottom: none; }
  .alert-sensor { color: var(--red); font-weight: 700; }
  .alert-lokasi { color: var(--muted); }
  .alert-pesan  { color: var(--text); }
  .alert-ts     { color: var(--muted); text-align: right; }

  .alert-empty {
    text-align: center; padding: 40px;
    color: var(--muted); font-size: .8rem;
  }

  /* ── Export button ──────────────────────────────── */
  .btn-export {
    display: inline-flex; align-items: center; gap: 8px;
    padding: 10px 20px; border-radius: 8px;
    background: linear-gradient(135deg, var(--accent), #0097a7);
    color: #000; font-family: var(--font-mono);
    font-size: .75rem; font-weight: 700;
    text-decoration: none; border: none; cursor: pointer;
    transition: all .2s; letter-spacing: .05em;
  }
  .btn-export:hover { transform: translateY(-2px); box-shadow: 0 8px
24px rgba(0,229,255,.3); }
  .btn-export:active { transform: translateY(0); }

  /* ── Loading / Error ────────────────────────────── */
  .loading {
    text-align: center; padding: 60px;
```

```
    color: var(--muted); font-size: .85rem;
  }
  .loading::after {
    content: '...';
    animation: dots 1.5s steps(3) infinite;
  }
  @keyframes dots {
    0%   { content: ''; }
    33%  { content: '.'; }
    66%  { content: '..'; }
    100% { content: '...'; }
  }

  .error-banner {
    background: rgba(255,23,68,.1); border: 1px solid var(--red);
    border-radius: 8px; padding: 14px 18px;
    font-size: .78rem; color: var(--red);
    display: none;
  }
  .error-banner.show { display: block; }

  /* ─── Footer ──────────────────────────────────────── */
  footer {
    position: relative; z-index: 5;
    text-align: center; padding: 20px;
    font-size: .65rem; color: var(--muted);
    border-top: 1px solid var(--border);
  }

  /* Responsive */
  @media (max-width: 768px) {
    main { padding: 16px; }
    header { padding: 14px 16px; }
    .chart-grid { grid-template-columns: 1fr; }
    .alert-row { grid-template-columns: 1fr 1fr; font-size: .65rem; }
    .alert-pesan { display: none; }
  }
</style>
</head>
```

```
<body>

<header>
  <div class="logo-area">
    <div class="logo-icon">❄</div>
    <div>
      <h1><span>Cold</span> Storage Monitor</h1>
      <div class="header-meta">Kelompok 6 · DB: kelompok6 · MQTT:
broker.hivemq.com</div>
    </div>
  </div>
  <div class="refresh-status">
    <div class="live-dot" id="live-dot"></div>
    <span id="status-text">Menghubungkan...</span>
    <span id="last-update">—</span>
  </div>
</header>

<div id="countdown-bar">
  <div id="countdown-fill" style="width:100%"></div>
</div>

<main>

  <!-- Error Banner -->
  <div class="error-banner" id="error-banner">
    ⚠ Tidak dapat terhubung ke server API. Pastikan
<code>api_server.py</code> sudah berjalan di
<code>localhost:5000</code>.
  </div>

  <!-- Stat Cards -->
  <div class="stat-grid" id="stat-grid">
    <div class="stat-card" style="--accent-card:#00e676">
      <div class="stat-label">Total Data 24j</div>
      <div class="stat-value" id="stat-total">—</div>
      <div class="stat-sub">dokumen tersimpan</div>
    </div>
    <div class="stat-card" style="--accent-card:#ff1744">
```

```
      <div class="stat-label">Alert 24j</div>
      <div class="stat-value" id="stat-alerts">—</div>
      <div class="stat-sub">anomali terdeteksi</div>
    </div>
    <div class="stat-card" style="--accent-card:#00e5ff">
      <div class="stat-label">Avg Suhu</div>
      <div class="stat-value" id="stat-suhu">—</div>
      <div class="stat-sub">°C rata-rata 24 jam</div>
    </div>
    <div class="stat-card" style="--accent-card:#ffd740">
      <div class="stat-label">Avg Kelembapan</div>
      <div class="stat-value" id="stat-hum">—</div>
      <div class="stat-sub">% rata-rata 24 jam</div>
    </div>
    <div class="stat-card" style="--accent-card:#ff6b35">
      <div class="stat-label">Sensor Bahaya</div>
      <div class="stat-value" id="stat-danger">—</div>
      <div class="stat-sub">unit status merah</div>
    </div>
  </div>

  <!-- Sensor Status -->
  <div>
    <div class="section-head">⬡ Status Sensor Real-time</div>
    <div class="sensor-grid" id="sensor-grid">
      <div class="loading">Memuat data sensor</div>
    </div>
  </div>

  <!-- Charts -->
  <div>
    <div class="section-head">📈 Tren 24 Jam Terakhir</div>
    <div class="chart-grid">
      <div class="chart-card">
        <div class="chart-title">🌡️ Suhu (°C)</div>
        <div class="chart-wrap">><canvas id="chart-suhu"></canvas></div>
      </div>
      <div class="chart-card">
        <div class="chart-title">💧 Kelembapan (%)</div>
```

```
        <div class="chart-wrap"><canvas
id="chart-kelembapan"></canvas></div>
      </div>
      <div class="chart-card">
        <div class="chart-title">☁ Amonia / NH₃ (ppm)</div>
        <div class="chart-wrap"><canvas
id="chart-amonia"></canvas></div>
      </div>
      <div class="chart-card">
        <div class="chart-title">🔵 Tekanan (Pa)</div>
        <div class="chart-wrap"><canvas
id="chart-tekanan"></canvas></div>
      </div>
    </div>
  </div>

  <!-- Alerts -->
  <div>
    <div class="section-head">🔴 Log Alert Terbaru</div>
    <div class="alert-box">
      <div class="alert-head">
        <div style="font-size:.75rem;color:var(--muted)">20 alert
terbaru ditampilkan</div>
        <a class="btn-export" href="/api/export/csv" download>
          ⬇ Export CSV 24 Jam
        </a>
      </div>
      <div class="alert-list" id="alert-list">
        <div class="loading">Memuat log alert</div>
      </div>
    </div>
  </div>

</main>

<footer>
  Cold Storage Monitoring System · Kelompok 6 · Auto-refresh setiap 10
detik
</footer>
```

```
<script>
// ─── Config ──────────────────────────────────────────────────────────────
const API_BASE      = window.location.origin;   // http://localhost:5000
const REFRESH_MS    = 10000;
let   countdownInt  = null;
let   refreshTimer  = null;
const charts        = {};

// ─── Chart.js default ────────────────────────────────────────────────────
Chart.defaults.color          = '#4a5568';
Chart.defaults.borderColor    = '#1e2d42';
Chart.defaults.font.family    = "'JetBrains Mono', monospace";
Chart.defaults.font.size      = 11;

function mkChart(id, label1, label2, color1, color2) {
  const ctx = document.getElementById(id).getContext('2d');
  return new Chart(ctx, {
    type: 'line',
    data: {
      labels: [],
      datasets: [
        {
          label: label1,
          data: [], borderColor: color1,
          backgroundColor: color1 + '18',
          borderWidth: 2, pointRadius: 3, tension: 0.4, fill: true,
        },
        {
          label: label2,
          data: [], borderColor: color2,
          backgroundColor: color2 + '18',
          borderWidth: 2, pointRadius: 3, tension: 0.4, fill: true,
        }
      ]
    },
    options: {
```

```
      responsive: true, maintainAspectRatio: false,
      animation: { duration: 600 },
      plugins: { legend: { position: 'top', labels: { boxWidth: 10,
padding: 16 } } },
      scales: {
        x: { grid: { color: '#1e2d4260' }, ticks: { maxTicksLimit: 12 }
},
        y: { grid: { color: '#1e2d4260' } }
      }
    }
  });
}

// Inisialisasi 4 chart
charts.suhu       = mkChart('chart-suhu',       'Cold Room A', 'Cold
Room B', '#00e5ff', '#ff6b35');
charts.kelembapan = mkChart('chart-kelembapan', 'Cold Room A', 'Cold
Room B', '#00e676', '#ffd740');
charts.amonia     = mkChart('chart-amonia',     'Cold Room A', 'Cold
Room B', '#b388ff', '#ff80ab');
charts.tekanan    = mkChart('chart-tekanan',    'Cold Room A', 'Cold
Room B', '#40c4ff', '#69f0ae');

// ─── Helpers
──────────────────────────────────────────────────────────────────────
function statusFromPm25(pm25) {
  if (pm25 === null || pm25 === undefined) return { cls:
'badge-unknown', label: '? N/A',      card: ''          };
  if (pm25 > 150)                          return { cls: 'badge-danger',
label: '🔴 Bahaya',  card: 'status-danger'  };
  if (pm25 > 50)                           return { cls:
'badge-warning', label: '⚠ Waspada', card: 'status-warning' };
  return                                          { cls: 'badge-normal',
label: '✅ Normal',   card: 'status-normal'  };
}

function sensorValue(doc) {
  const t = doc.tipe_sensor;
  if (t === 'suhu')       return { val: doc.suhu      != null ?
```

```
`${doc.suhu}°C`        : '—', icon: '🌡️' };
  if (t === 'kelembapan') return { val: doc.kelembapan!= null ?
`${doc.kelembapan}%`    : '—', icon: '💧' };
  if (t === 'amonia')     return { val: doc.amonia     != null ?
`${doc.amonia} ppm`    : '—', icon: '☁️'  };
  if (t === 'pintu')      return { val: doc.status_pintu === 1 ?
'TERBUKA 🚪'           : 'Tertutup', icon: '🚪' };
  if (t === 'tekanan')    return { val: doc.tekanan    != null ?
`${doc.tekanan} Pa`    : '—', icon: '🔵' };
  return { val: '—', icon: '📡' };
}

function fmtTs(ts) {
  if (!ts) return '—';
  try { return new Date(ts).toLocaleString('id-ID', { hour12: false });
}
  catch { return ts.slice(0, 19).replace('T',' '); }
}

// ─── Render sensor cards
──────────────────────────────────────────────────────────────────────
function renderSensors(sensors) {
  const grid = document.getElementById('sensor-grid');
  grid.innerHTML = sensors.map(doc => {
    const st = statusFromPm25(doc.pm25);
    const sv = sensorValue(doc);
    return `
      <div class="sensor-card ${st.card}">
        <div class="sensor-top">
          <div>
            <div class="sensor-id">${sv.icon} ${doc.sensor_id ||
'—'}</div>
            <div class="sensor-lokasi">${doc.lokasi || '—'}</div>
          </div>
          <span class="sensor-badge ${st.cls}">${st.label}</span>
        </div>
        <div class="sensor-value">${sv.val}</div>
        <div class="sensor-tipe">${doc.tipe_sensor || '—'} ·
PM2.5≈${doc.pm25 ?? '—'}</div>
```

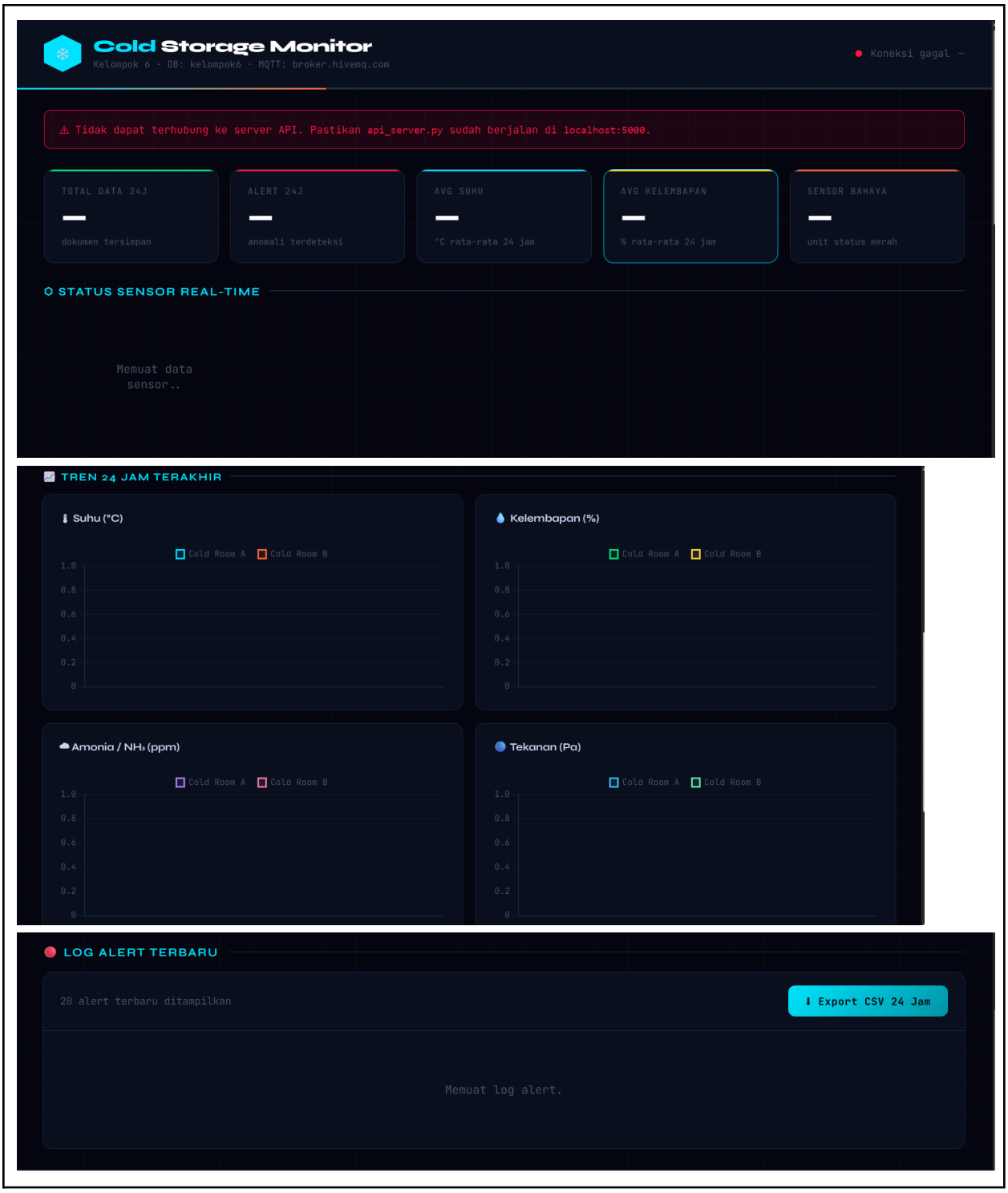
```
        <div class="sensor-ts">⏱ ${fmtTs(doc.timestamp)}</div>
      </div>
    `;
  }).join('');
}

// ─── Render alerts ──────────────────────────────────────────────────────
function renderAlerts(alerts) {
  const list = document.getElementById('alert-list');
  if (!alerts.length) {
    list.innerHTML = '<div class="alert-empty">✅ Tidak ada alert dalam
20 data terakhir</div>';
    return;
  }
  list.innerHTML = alerts.map(a => `
    <div class="alert-row">
      <div class="alert-sensor">🔴 ${a.sensor_id || '—'}</div>
      <div class="alert-lokasi">${a.lokasi || '—'}</div>
      <div class="alert-pesan">${a.pesan || '—'}</div>
      <div class="alert-ts">${fmtTs(a.timestamp)}</div>
    </div>
  `).join('');
}

// ─── Update charts ──────────────────────────────────────────────────────
function updateChart(chartObj, data) {
  const lokasiA = 'Cold Room A';
  const lokasiB = 'Cold Room B';
  const ptsA = data[lokasiA] || [];
  const ptsB = data[lokasiB] || [];

  // Buat label jam 0-23
  const allHours = [...new Set([...ptsA, ...ptsB].map(p =>
p.hour))].sort((a,b)=>a-b);
  const labels   = allHours.map(h => `${String(h).padStart(2,'0')}:00`);

  const mapByHour = pts => {
```

```
    const m = {};
    pts.forEach(p => m[p.hour] = p.value);
    return allHours.map(h => m[h] ?? null);
  };

  chartObj.data.labels          = labels;
  chartObj.data.datasets[0].data = mapByHour(ptsA);
  chartObj.data.datasets[1].data = mapByHour(ptsB);
  chartObj.update('active');
}

// ─── Stat cards ──────────────────────────────────────────────────────────
function updateStats(summary, totalAlerts) {
  document.getElementById('stat-total').textContent   =
summary.total_data_24h?.toLocaleString('id-ID') ?? '—';
  document.getElementById('stat-alerts').textContent  =
summary.total_alerts_24h ?? '—';
  document.getElementById('stat-suhu').textContent    = summary.avg_suhu
!= null ? `${summary.avg_suhu}` : '—';
  document.getElementById('stat-hum').textContent     =
summary.avg_kelembapan != null ? `${summary.avg_kelembapan}` : '—';
  document.getElementById('stat-danger').textContent  =
summary.total_danger ?? '—';
}

// ─── Fetch all ───────────────────────────────────────────────────────────
async function fetchAll() {
  const dot  = document.getElementById('live-dot');
  const txt  = document.getElementById('status-text');
  const err  = document.getElementById('error-banner');

  try {
    const [resStatus, resChart, resAlerts, resSummary] = await
Promise.all([
      fetch(`${API_BASE}/api/status`),
      fetch(`${API_BASE}/api/chart`),
      fetch(`${API_BASE}/api/alerts?limit=20`),
```

```
      fetch(`${API_BASE}/api/summary`),
    ]);

    if (!resStatus.ok) throw new Error(`HTTP ${resStatus.status}`);

    const [dStatus, dChart, dAlerts, dSummary] = await Promise.all([
      resStatus.json(), resChart.json(), resAlerts.json(),
resSummary.json()
    ]);

    // Sensors
    renderSensors(dStatus.sensors || []);

    // Charts
    ['suhu','kelembapan','amonia','tekanan'].forEach(t => {
      if (dChart[t]) updateChart(charts[t], dChart[t]);
    });

    // Alerts
    renderAlerts(dAlerts.alerts || []);

    // Stats
    updateStats(dSummary, dStatus.total_alerts);

    // UI ok
    dot.className    = 'live-dot';
    txt.textContent  = 'Live';
    err.classList.remove('show');
    document.getElementById('last-update').textContent =
      new Date().toLocaleTimeString('id-ID');

  } catch (e) {
    dot.className   = 'live-dot error';
    txt.textContent = 'Koneksi gagal';
    err.classList.add('show');
    console.error('[Monitor]', e);
  }
}
```

```
// ─── Countdown bar ──────────────────────────────────────────────────────────
let countdownMs = REFRESH_MS;
function startCountdown() {
  clearInterval(countdownInt);
  countdownMs = REFRESH_MS;
  const fill = document.getElementById('countdown-fill');
  countdownInt = setInterval(() => {
    countdownMs -= 200;
    fill.style.width = Math.max(0, (countdownMs / REFRESH_MS) * 100) +
'%';
    if (countdownMs <= 0) {
      countdownMs = REFRESH_MS;
      fill.style.width = '100%';
    }
  }, 200);
}

// ─── Auto refresh ───────────────────────────────────────────────────────────
async function fullRefresh() {
  await fetchAll();
  startCountdown();
}

fullRefresh();
refreshTimer = setInterval(fullRefresh, REFRESH_MS);
</script>
</body>
</html>
```

Output:



10. Buat file requirements.txt dan isi dengan di bawah ini:

```
# requirements.txt – Cold Storage IoT Monitoring System
# Install dengan: pip install -r requirements.txt
```

```
paho-mqtt>=1.6.1
pymongo>=4.6.0
matplotlib>=3.8.0
python-dotenv>=1.0.0
```

11. Buat file README.md lalu isi dengan kode berikut:

```
# 🧊 Cold Storage IoT Monitoring System

Sistem pemantauan kualitas lingkungan untuk **Cold Storage (ruang
penyimpanan daging)** berbasis arsitektur IoT dengan MQTT dan MongoDB.

---

## 👥 Pembagian Peran

| Anggota | Peran | File |
|---------|-------|------|
| Anggota 1 | MQTT Publisher Simulasi | `pub_udara.py` |
| Anggota 2 | MQTT Subscriber & Penyimpanan | `sub_udara.py` |
| Anggota 3 | Analisis & Dashboard | `analisis_dashboard.py` |
| Anggota 4 | Status Terkini & Dokumentasi | `status_udara.py` |

---

## 🌡️ Sensor yang Digunakan

| Sensor | ID | Variabel | Kondisi Bahaya |
|--------|-----|----------|----------------|
| Suhu (PT100/DS18B20) | TEMP-A1, TEMP-B1 | `suhu` °C | > -10°C |
| Kelembapan (DHT22) | HUM-A1, HUM-B1 | `kelembapan` % | < 70% |
| Gas Amonia (MQ-137) | NH3-A1, NH3-B1 | `amonia` ppm | > 25 ppm |
| Status Pintu (Magnetic Switch) | DOOR-A1, DOOR-B1 | `status_pintu` 0/1
| = 1 (terbuka) |
| Tekanan Diferensial | PRESS-A1, PRESS-B1 | `tekanan` Pa | < 20 Pa |

---

## ⚙️ Prasyarat
```

- Python 3.9+
- MongoDB (lokal atau Atlas)
- Mosquitto MQTT Broker

Install Mosquitto (Windows)

```
https://mosquitto.org/download/
```

Install Mosquitto (Linux/Mac)

```bash
sudo apt install mosquitto mosquitto-clients   # Ubuntu/Debian
brew install mosquitto                          # Mac
```

Start Mosquitto

```bash
mosquitto -v   # verbose mode
```

🚀 Cara Menjalankan Sistem

Langkah 1 – Clone & Install Dependencies

```bash
git clone <repo>
cd cold_storage_iot
pip install -r requirements.txt
```

Langkah 2 – Konfigurasi Environment

```bash
cp .env.example .env
# Edit .env sesuai konfigurasi MongoDB dan MQTT Anda
```

Langkah 3 – Inisialisasi Database MongoDB

```bash
python setup_mongodb.py
```

> Membuat database `kelompok6`, koleksi `kualitas_udara` dan `alert`, serta mengisi data historis 24 jam.

Langkah 4 – Jalankan Subscriber (Terminal 1)

```bash
python sub_udara.py
```

Langkah 5 – Jalankan Publisher (Terminal 2)

```bash
python pub_udara.py
```

Langkah 6 (Opsional) – Status Monitor (Terminal 3)

```bash
python status_udara.py
```

Langkah 7 (Opsional) – Buat Dashboard & Export CSV

```bash
python analisis_dashboard.py
```

Output: `dashboard_udara.png` dan `data_24jam.csv`

📁 Struktur File

```
cold_storage_iot/
├── pub_udara.py           # Anggota 1: Publisher MQTT
├── sub_udara.py           # Anggota 2: Subscriber + MongoDB
├── analisis_dashboard.py  # Anggota 3: Analisis & Grafik
├── status_udara.py        # Anggota 4: Status Monitor
├── setup_mongodb.py       # Setup & seed database
├── requirements.txt       # Dependensi Python
```

```
├── .env.example            # Template environment variables
└── README.md               # Dokumentasi ini
```

🗄️ Struktur Database MongoDB

Database: `kelompok6`

Koleksi `kualitas_udara`

```json
{
  "sensor_id":    "NH3-A1",
  "lokasi":       "Cold Room A",
  "tipe_sensor":  "amonia",
  "timestamp":    "2025-01-15T08:30:00+00:00",
  "suhu":         null,
  "kelembapan":   null,
  "amonia":       8.5,
  "status_pintu": null,
  "tekanan":      null,
  "pm25":         12.75
}
```

Koleksi `alert`

```json
{
  "sensor_id":   "NH3-A1",
  "lokasi":      "Cold Room A",
  "tipe_sensor": "amonia",
  "pm25":        210.0,
  "timestamp":   "2025-01-15T08:30:00+00:00",
  "pesan":       "Amonia berbahaya! NH₃=35.0 ppm",
  "created_at":  "2025-01-15T08:30:00+00:00"
}
```

```
---

## 🔗 Website Dashboard

Buka `index.html` di browser untuk melihat dashboard web interaktif.

---

## 📌 Catatan

- Nilai `pm25` pada sistem ini berfungsi sebagai **proxy universal anomali** untuk memudahkan deteksi lintas tipe sensor.
- Setiap sensor memiliki probabilitas **5%** untuk menghasilkan kondisi anomali dalam simulasi.
- Threshold bahaya: `pm25 > 150` (ekuivalen dengan kondisi berbahaya pada masing-masing sensor).
```

12. Buat file env.test lalu isi dengan kode berikut:

```
# ─── Environment Variables – Cold Storage IoT ──────────────────────────────
# Salin file ini menjadi .env lalu sesuaikan nilainya
# Di terminal: cp .env.example .env

# ── MQTT Broker ──────────────────────────────────────────────────────────────
MQTT_HOST=localhost
MQTT_PORT=1883
# Jika broker butuh autentikasi:
# MQTT_USERNAME=username_anda
# MQTT_PASSWORD=password_anda

# ── MongoDB ──────────────────────────────────────────────────────────────────
# Lokal tanpa autentikasi:
MONGO_URI=mongodb://localhost:27017/

# MongoDB Atlas (cloud) – ganti dengan connection string Anda:
# MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

```
# MongoDB lokal dengan autentikasi:
# MONGO_URI=mongodb://admin:password@localhost:27017/

# ── Nama Database
──────────────
```